

**Санкт-Петербургский государственный электротехнический
университет «ЛЭТИ» им. В.И. Ульянова (Ленина)**

Аналитическая обработка данных в задачах информационной безопасности

Санкт-Петербург
2026

Содержание

Глоссарий	5
1. Лекционный курс	7
1.1. Лекция 1: Основы ClickHouse	7
1.1.1. Определение	7
1.1.2. OLAP vs OLTP	7
1.1.3. Когда нужен ClickHouse	7
1.1.4. Взаимодействие с ClickHouse	7
1.1.5. Движки таблиц (Engines)	8
1.1.5.1. Основные группы движков	8
1.1.5.2. Семейство MergeTree	8
1.1.6. Merge — фоновый процесс объединения данных	8
1.1.7. Партиционирование	8
1.1.7.1. Типичные ошибки партиционирования	8
1.1.8. TTL — управление жизненным циклом данных	9
1.1.9. ORDER BY — физический порядок хранения	9
1.1.9.1. Проектирование ORDER BY	9
1.1.10. PRIMARY KEY	10
1.1.11. Проблема дублей и ReplacingMergeTree	10
1.1.11.1. Способы получить дедуплицированные данные	10
1.2. Лекция 2: RBAC и ABAC: модели управления доступом в ИБ	12
1.2.1. Зачем нужны модели управления доступом	12
1.2.2. RBAC: роль как единица администрирования	12
1.2.3. Иерархии ролей и разделение обязанностей	12
1.2.4. ABAC: политика на основе атрибутов	12
1.2.5. Как работает проверка доступа в ABAC	13
1.2.6. RBAC и ABAC: сравнение	13
1.2.7. Комбинированный подход RBAC + ABAC	13
1.2.8. Типичные ошибки проектирования доступа	13
1.3. Лекция 3: Распределённый ClickHouse: шарды, реплики и координация	14
1.3.1. Координация: ZooKeeper и ClickHouse Keeper	14
1.3.2. Шарды и реплики	14
1.3.3. Локальные и распределённые таблицы	14
1.3.4. Путь записи и чтения	14
1.3.5. Операционные риски распределённого кластера	15
1.3.6. Практические ориентиры проектирования	15
1.4. Лекция 4: Обезличивание персональных данных: модели и техники	16
1.4.1. Регуляторный контекст	16
1.4.2. Квази-идентификаторы	16
1.4.3. k-анонимность	16
1.4.4. Основные техники обезличивания	16
1.4.5. Обезличивание в OLAP-контексте	17
1.5. Лекция 5: Materialized Views: инкрементальная агрегация	18
1.5.1. VIEW, Materialized View и Table — три разных объекта	18
1.5.2. Главное правило: разберитесь, как работает <i>ваш</i> MV	18
1.5.3. Архитектура MV в ClickHouse: три объекта	19
1.5.4. Выбор движка целевой таблицы	19
1.5.5. Как MV обрабатывает данные	19
1.5.6. Когда Materialized View подходит	20
1.5.7. Когда Materialized View не подходит	20
1.5.7.1. Динамическое ранжирование (top-N)	20
1.5.7.2. Сравнение с историческим базовым уровнем	20

1.5.7.3.	Оконная и позиционная логика	20
1.5.8.	Сырые данные и агрегаты: разные роли	20
1.5.9.	Операционные особенности	21
1.5.10.	Типичные ошибки при работе с MV	21
1.6.	Лекция 6: Private Set Intersection: идея, модель угроз и упрощённый DH-протокол	22
1.6.1.	Зачем нужен PSI	22
1.6.2.	Почему обычного хеширования недостаточно	22
1.6.3.	Модель угроз: что защищает PSI, а что нет	22
1.6.4.	Идея PSI: узнать пересечение без раскрытия полных множеств	22
1.6.5.	Упрощённый протокол на основе Diffie-Hellman	23
1.6.6.	Почему эта схема работает	23
1.6.7.	Архитектурная схема: ClickHouse хранит данные, Python выполняет криптографию	24
1.6.8.	Ограничения и компромиссы PSI	24
1.7.	Лекция 7: Выбор подхода для приватного сопоставления и совместной аналитики	25
1.7.1.	Сначала формулируем задачу, а не выбираем инструмент	25
1.7.2.	Обычный JOIN: когда доверенная среда уже есть	25
1.7.3.	Обезличивание и псевдонимизация: когда достаточно трансформации данных	25
1.7.4.	PSI: когда нужен только факт пересечения	25
1.7.5.	TEE: когда нужен более общий вычислительный сценарий	26
1.7.6.	Сравнение подходов	26
1.7.7.	Практические сценарии выбора	26
1.7.8.	Типичные ошибки архитектурного выбора	27
2.	Лабораторный практикум	28
2.1.	Введение в лабораторный практикум	28
2.2.	Лабораторная работа 1: RBAC в ClickHouse	29
2.2.1.	Цель	29
2.2.2.	Подготовка	29
2.2.3.	Часть 1: Роли и привилегии	29
2.2.3.1.	Задание	29
2.2.3.2.	Контрольные вопросы	30
2.2.4.	Часть 2: Row Policies	30
2.2.4.1.	Задание	30
2.2.4.2.	Контрольные вопросы	30
2.2.5.	Часть 3: Квоты и ограничения	30
2.2.5.1.	Задание	30
2.2.5.2.	Контрольные вопросы	30
2.2.6.	Часть 4: Проверка модели доступа	31
2.2.6.1.	Задание	31
2.2.6.2.	Вывод	31
2.3.	Лабораторная работа 2: Обезличивание персональных данных	32
2.3.1.	Цель	32
2.3.2.	Подготовка данных	32
2.3.3.	Вариант 1: Мутация (ALTER UPDATE)	33
2.3.3.1.	Задание	33
2.3.3.2.	Контрольные вопросы	33
2.3.3.3.	Когда уместно, когда нет	33
2.3.4.	Вариант 2: ReplacingMergeTree	33
2.3.4.1.	Задание	33
2.3.4.2.	Контрольные вопросы	34
2.3.4.3.	Когда уместно, когда нет	34
2.3.5.	Вариант 3: ETL-пайплайн	34
2.3.5.1.	Задание	34
2.3.5.2.	Контрольные вопросы	35

2.3.5.3.	Когда уместно, когда нет	35
2.3.6.	Проверка k-анонимности	35
2.3.6.1.	Задание	35
2.3.6.2.	Контрольные вопросы	36
2.3.7.	Итоговое сравнение	36
2.4.	Лабораторная работа 3: Аудит логов безопасности	37
2.4.1.	Цель	37
2.4.2.	Часть 1: Схема данных и генерация событий	37
2.4.3.	Часть 2: Materialized View — автоматическая агрегация	37
2.4.3.1.	Задание	37
2.4.3.2.	Контрольные вопросы	38
2.4.4.	Часть 3: Brute-force detection — запрос к сырым данным vs. агрегат	38
2.4.4.1.	Задание	38
2.4.4.2.	Контрольные вопросы	39
2.4.5.	Часть 4: Границы применимости Materialized View	39
2.4.5.1.	Задание	39
2.4.5.2.	Почему так произошло	40
2.4.5.3.	Правильный подход	40
2.4.5.4.	Контрольные вопросы	40
2.4.6.	Часть 5: Append-only и границы его полезности для безопасности	40
2.4.6.1.	Задание	40
2.4.6.2.	Где это помогает, а где нет	41
2.4.6.3.	Контрольные вопросы	41
2.4.7.	Часть 6: Обнаружение аномальной активности	41
2.4.7.1.	Задание	41
2.4.7.2.	Контрольные вопросы	42
2.4.8.	Часть 7 (повышенная сложность): Дополнительные задачи	42
2.5.	Лабораторная работа 4: Private Set Intersection	43
2.5.1.	Цель	43
2.5.2.	Часть 1: Подготовка данных	43
2.5.2.1.	Задание	43
2.5.2.2.	Контрольные вопросы	44
2.5.3.	Часть 2: Реализация PSI	44
2.5.3.1.	Задание	44
2.5.3.2.	Контрольные вопросы	45
2.5.4.	Часть 3: Верификация	45
2.5.4.1.	Задание	45
2.5.4.2.	Контрольные вопросы	46
2.5.5.	Связь с архитектурой ClickHouse	46

Глоссарий

Ad-hoc анализ — Разовые, незапланированные аналитические запросы для исследования данных или проверки гипотез. В отличие от регулярных отчётов, выполняются по требованию без предварительной настройки ETL-пайплайнов. Требуется интерактивной производительности и гибкого SQL.

API — Application Programming Interface — программная абстракция, определяющая методы и протоколы взаимодействия между программными компонентами. Обеспечивает унифицированный доступ к функциональности системы посредством стандартизированных вызовов, скрывая детали внутренней реализации.

BI — Business Intelligence — технологии и практики сбора, интеграции, анализа и представления бизнес-информации. Включает инструменты для создания отчётов, дашбордов, визуализации данных и поддержки принятия управленческих решений на основе данных.

CDN — Content Delivery Network — географически распределённая сеть серверов для кэширования и доставки статического контента пользователям с минимальной задержкой.

Компонент — Единица вычисления либо хранения данных, имеющая явно определённый контракт взаимодействия (интерфейс) и набор зависимостей.

CPU-time — Среднее процессорное время на транзакцию; оценивается через число ядер и TPS (мс на транзакцию).

CRM — Customer Relationship Management — система управления взаимоотношениями с клиентами. Программное обеспечение для автоматизации стратегий взаимодействия с клиентами, включая продажи, маркетинг, техподдержку и аналитику клиентских данных.

DAU — Daily Active Users — количество уникальных пользователей, взаимодействовавших с системой за сутки.

СУБД — Система управления базами данных — комплекс программных средств для создания, модификации и управления данными в базе данных. Обеспечивает структурированное хранение, эффективный доступ, целостность данных и управление конкурентным доступом.

DML — Data Manipulation Language — операции над данными в БД: SELECT/INSERT/UPDATE/DELETE.

downstream — В контексте конкретного вызова — вызываемая сторона (зависимость), которая обрабатывает запрос, полученный от upstream-сервиса.

Эмбединг — Векторное представление объекта (текста, изображения, пользователя, товара), полученное моделью машинного обучения для измерения семантической близости через расстояния в векторном пространстве.

Индекс — Вспомогательная структура данных в системах управления базами данных, предназначенная для оптимизации производительности операций поиска и извлечения информации. Организует упорядоченное представление данных для существенного сокращения времени доступа к записям.

Ввод-вывод — Input/Output — совокупность механизмов и протоколов обмена данными между вычислительной системой и внешними устройствами или другими системами. Включает операции получения данных из источников и передачи результатов обработки, представляя критически важный компонент архитектуры компьютерных систем.

IOPS — I/O Operations Per Second — количество операций ввода-вывода (чтения/записи) в секунду на уровне диска.

IoT — Internet of Things — сетевое взаимодействие физических устройств с датчиками и контроллерами. Характеризуется высокой частотой передачи небольших порций данных (телеметрия, метрики).

Слой — Множество компонентов, объединённых общей ответственностью и уровнем абстракции, и участвующих в системе согласно установленным правилам межслойных зависимостей. Говорят что слой А зависит от слоя Б если хотя бы один компонент слоя А зависит от компонента слоя Б.

LLM — Large Language Model — большая языковая модель на основе нейронных сетей, обученная на огромных объёмах текстовых данных. Способна генерировать код, текст, отвечать на вопросы и выполнять задачи программирования на основе естественно-языковых промптов.

LOC — Lines of Code — метрика объёма программного кода, измеряемая количеством строк. Используется для оценки размера кодовой базы, сложности проекта и трудозатрат на разработку. Не включает пустые

строки и комментарии при подсчёте физических строк (PLOC), либо учитывает только исполняемые инструкции при подсчёте логических строк (LLOC).

MAU — Monthly Active Users — количество уникальных пользователей за месяц.

Метаданные — Структурированная информация, описывающая свойства других данных: время создания, размер, тип, владелец, права доступа. Используется для индексирования и маршрутизации без чтения основного содержимого.

Морселизация — Техника параллельной обработки данных путём разбиения на небольшие фрагменты (morsels) фиксированного размера (обычно ~1024 строки). Каждый morsel обрабатывается независимо векторизованным движком, что обеспечивает эффективное использование CPU-кэшей и параллелизацию на уровне потоков.

OLAP — Online Analytical Processing — класс систем для аналитической обработки данных. Ориентирован на выполнение сложных агрегирующих запросов по большим объёмам данных, использует колоночное хранение для оптимизации производительности аналитики.

OLTP — Online Transaction Processing — класс систем обработки транзакций в реальном времени. Характеризуется большим количеством коротких транзакций (чтение, вставка, обновление, удаление), строковым хранением данных и требованиями к консистентности.

QPS — Queries Per Second — количество запросов к базе данных в секунду. Один HTTP-запрос может порождать несколько запросов к БД.

RPS — Requests Per Second — количество пользовательских запросов (например HTTP) в секунду.

Селективность — Свойство условия, индекса или колонки хорошо отбирать небольшую долю строк. Чем выше селективность, тем меньше записей подходит под условие и тем полезнее такой признак для точечного поиска.

Сессия — Логический контекст взаимодействия клиента с системой между множественными запросами. Хранит состояние пользователя: аутентификацию, корзину, настройки. Реализуется через cookies + серверное хранилище (Redis) или JWT-токены.

SLA — Service Level Agreement — соглашение об уровне обслуживания с количественными метриками: uptime (99.9% = 8.76 ч простоя/год), задержка (p99 < 200 мс).

Разреженный индекс — Индекс, который хранит не запись для каждой строки, а опорные значения только для крупных блоков данных. Он занимает меньше места и позволяет быстро отбросить неподходящие блоки, но обычно не даёт такой точности, как индекс по каждой строке.

Support — Служба поддержки и обработка обращений пользователей: тикеты, инциденты, запросы на возврат и консультации.

TPS — Transactions Per Second — количество транзакций БД в секунду.

Транзакция — Последовательность операций чтения и записи в базе данных, рассматриваемая как единая логическая единица работы. Гарантирует атомарность выполнения: либо все операции завершаются успешно и фиксируются (COMMIT), либо при ошибке все изменения откатываются (ROLLBACK).

upstream — В контексте конкретного вызова — вызывающая сторона (клиент), которая инициирует запрос к другому сервису и ожидает результат.

1. Лекционный курс

1.1. Лекция 1: Основы ClickHouse

Эта лекция формирует базовый понятийный аппарат курса: где находится место ClickHouse в архитектуре данных, как устроено хранение в MergeTree и почему в OLAP-системах критичны правильные ключи сортировки, партиционирование и фоновые merge-процессы.

1.1.1. Определение

ClickHouse — это распределённая СУБД с открытым исходным кодом, созданная для аналитической обработки запросов (OLAP) и хранения больших объёмов данных. Изначально разработана в Яндексе для Яндекс.Метрики — системы веб-аналитики, обрабатывающей миллиарды событий в сутки.

ClickHouse оптимизирован для сложных аналитических запросов, дашбордов и отчётности с низкой латентностью. Это позволяет аналитикам и инженерам строить интерактивные витрины и системы мониторинга поверх петабайтных объёмов данных, сохраняя высокую производительность даже при большом числе конкурентных запросов.

1.1.2. OLAP vs OLTP

	OLTP	OLAP
Назначение	Быстрые короткие транзакции: вставка, обновление, чтение отдельных записей	Сложные аналитические запросы: агрегация, группировка, анализ больших объёмов исторических данных
Оптимизация	Минимальная задержка, высокая согласованность при большом числе одновременных пользователей	Максимальная пропускная способность, высокая скорость сканирования миллионов/миллиардов записей
Схемы	Нормализованные, строгие ограничения, строковое хранение	Денормализованные (звезда, снежинка), колоночное хранение
Примеры	Платежи, заказы, CRM, банковские транзакции	BI, дашборды, Ad-hoc анализ запросы, мониторинг

Таблица 1. Сравнение OLTP и OLAP

1.1.3. Когда нужен ClickHouse

Подходит:

- Аналитика на больших объёмах данных (шардирование, горизонтальное масштабирование).
- Высокая скорость агрегирования.
- Интерактивные дашборды и мониторинг.
- Поточковая обработка — анализ данных в реальном времени.

Не подходит:

- Нужны полноценные транзакции (многотабличные ACID-транзакции, сложные BEGIN/COMMIT/ROLLBACK сценарии). Экспериментальная поддержка транзакций существует, но ограничена.
- Маленькие датасеты (для этого есть DuckDB).

1.1.4. Взаимодействие с ClickHouse

ClickHouse поддерживает несколько протоколов доступа:

- **Нативный TCP-протокол** — максимальная производительность, используется clickhouse-client и большинством драйверов.
- **HTTP / HTTPS интерфейс** — удобен для интеграции с веб-приложениями и мониторингом.
- **MySQL / PostgreSQL wire protocol** — совместимость с существующими клиентами и инструментами.
- **JDBC / ODBC драйверы** — для Java-приложений и BI-инструментов.

Вне зависимости от выбранного протокола, общение происходит на обычном SQL:

```
SELECT t1.id, t2.name
FROM t1
JOIN t2 ON t1.id = t2.id
```

1.1.5. Движки таблиц (Engines)

Поведение таблицы полностью определяется выбранным движком (engine). Именно engine определяет:

- как данные физически хранятся на диске,
- можно ли выполнять merge,
- поддерживается ли репликация,
- как работает дедупликация,
- возможны ли TTL и мутации (UPDATE/DELETE).

Выбор engine — это архитектурное решение. Его нельзя легко изменить без полной пересборки таблицы.

1.1.5.1. Основные группы движков

MergeTree-семейство — используется в большинстве продакшн-кейсов. Предназначено для больших объёмов аналитических данных.

Log / TinyLog — простые движки без merge и индексов. Используются для временных таблиц и тестирования.

Distributed — логический движок для работы с шардами. Сам данные не хранит.

Special engines — Kafka, Buffer, Dictionary — используются для интеграций и потоковой обработки.

1.1.5.2. Семейство MergeTree

- **MergeTree** — базовый вариант.
- **ReplicatedMergeTree** — с репликацией между нодами.
- **ReplacingMergeTree** — дедупликация по ключу при merge.
- **SummingMergeTree** — агрегация числовых колонок при merge.
- **AggregatingMergeTree** — хранение агрегатных состояний.
- **Collapsing / VersionedCollapsingMergeTree** — хранение событий с отменами.

1.1.6. Merge — фоновый процесс объединения данных

INSERT в ClickHouse никогда не переписывает существующие данные — он создаёт новые части (parts). Merge — это фоновый процесс, который объединяет эти части.

Это не оптимизация, а фундаментальная часть архитектуры. Система предполагает, что:

- данные пишутся быстро (append-only),
- порядок и консистентность достигаются позже (при merge),
- аналитика допускает небольшую задержку в «идеальности» данных.

1.1.7. Партиционирование

Партиции — это логическое разбиение таблицы на независимые части (обычно по дате), задаваемое выражением PARTITION BY.

Партиции позволяют:

- быстро удалять данные (DROP PARTITION — мгновенная операция),
- ограничивать область merge,
- эффективно применять TTL,
- ускорять запросы с фильтрами по ключу партиционирования.

1.1.7.1. Типичные ошибки партиционирования

Основная ошибка — слишком мелкие партиции (например, партиционирование по user_id или uuid).

Количество партиций	Оценка
Десятки – сотни	Норма
1–2 тысячи	Допустимо в редких случаях
Десятки тысяч	Плохо
Сотни тысяч и более	Катастрофа

Таблица 2. Рекомендации по количеству партиций на таблицу

1.1.8. TTL — управление жизненным циклом данных

TTL — механизм автоматического удаления или перемещения данных по времени без внешних задач. Работает только в MergeTree-движках.

Важно: TTL не удаляет данные сразу, а дожидается merge. Данные будут существовать дольше указанного срока. При необходимости можно вручную запустить `OPTIMIZE TABLE ... FINAL`.

TTL работает на уровне частей (parts), а не партиций. Если TTL совпадает с границей партиций и включена настройка `ttl_only_drop_parts = 1`, ClickHouse целиком удаляет части, в которых все строки истекли — это быстрая операция. Без этой настройки (по умолчанию выключена) удаляются отдельные строки через мутации, что значительно дороже.

```
CREATE TABLE events_optimized (
    event_date Date,
    user_id UInt64,
    event_type String,
    data String
) ENGINE = MergeTree()
PARTITION BY toYYYYMM(event_date)
ORDER BY (event_date, user_id)
TTL event_date + INTERVAL 3 MONTH;
```

1.1.9. ORDER BY — физический порядок хранения

`ORDER BY` в определении таблицы определяет физический порядок хранения данных на диске. Это **не** порядок вывода в `SELECT` — для этого нужен отдельный `ORDER BY` в запросе.

Если поле из `WHERE` совпадает с префиксом `ORDER BY`, ClickHouse:

- читает минимальное количество данных,
- эффективно использует min/max индексы,
- эффективно использует разреженный индекс,
- существенно уменьшает I/O.

Разберём эти механизмы отдельно.

Min/max индекс хранит для каждого крупного фрагмента минимум и максимум значения. Например, если для некоторого фрагмента записано `2024-01-01 ... 2024-01-31`, то запрос `WHERE event_date = '2024-03-10'` сразу понимает, что этот фрагмент читать не нужно. Такой индекс хорошо работает для дат, чисел и других колонок, где условие задаёт диапазон.

Разреженный индекс — это основной индекс MergeTree. Он хранит не каждую строку, а только опорные значения для крупных блоков данных. Обычно группа содержит до 8192 строк. По этим опорным значениям ClickHouse быстро находит, какие группы могут содержать нужные строки, и читает только их. Поэтому `ORDER BY` в ClickHouse так важен: именно он определяет, насколько полезным будет этот индекс.

1.1.9.1. Проектирование ORDER BY

Главное правило: **первыми ставить колонки с малым числом различных значений** (низкой кардинальностью). Здесь пригодятся два термина:

- **Кардинальность** — число различных значений в колонке. У `event_type` — десятки, у `user_id` — миллионы.

- **Селективность условия** — доля строк, которые оно возвращает. Условие `status = 'error'` с долей 0.01 отбирает 1% строк — высокая селективность. Условие `country = 'RU'` с долей 0.6 — низкая.

В OLTP-системах принято обратное: сначала идёт самая уникальная колонка, чтобы быстрее найти конкретную строку. В ClickHouse иначе: если первой стоит, скажем, `event_type` с десятком значений, строки одного типа лежат рядом и разреженный индекс легко отсекает ненужные куски. Если же первой поставить `user_id` с миллионами значений, данные перемешаны и индекс почти не помогает. Бонус: соседние строки похожи — данные сжимаются лучше.

```
-- Хорошо: event_type (5-10 значений) первым, затем user_id (миллионы)
ORDER BY (event_type, user_id, event_date);
```

```
-- Плохо: user_id первым -- высокая кардинальность ломает
-- эффективность разреженного индекса для последующих колонок
ORDER BY (user_id, event_type, event_date);
```

1.1.10. PRIMARY KEY

PRIMARY KEY определяет разреженный индекс и **ничего больше**.

Ключевые отличия от OLTP-систем:

- **ПК не гарантирует уникальность** — дубли разрешены.
- ПК обязан быть **префиксом** ORDER BY.
- Если PRIMARY KEY не указан — автоматически равен ORDER BY.

```
-- PRIMARY KEY автоматически = (timestamp, server_id, level)
ORDER BY (timestamp, server_id, level);
```

```
-- Можно указать более короткий ПК для экономии памяти:
ORDER BY (timestamp, server_id, level, request_id)
PRIMARY KEY (timestamp, server_id);
-- level и request_id участвуют в сортировке, но не в индексе
```

1.1.11. Проблема дублей и ReplacingMergeTree

Поскольку PRIMARY KEY не гарантирует уникальность, а UPSERT отсутствует, для управления дублями используется ReplacingMergeTree.

```
CREATE TABLE users_replacing (
    user_id UInt64,
    name String,
    email String,
    updated_at DateTime
) ENGINE = ReplacingMergeTree(updated_at)
PRIMARY KEY user_id;
```

При вставке строки с тем же ключом дубли **сохраняются**:

```
INSERT INTO users_replacing VALUES
    (1, 'Alice', 'alice@example.com', '2026-01-01 10:00:00');
INSERT INTO users_replacing VALUES
    (1, 'Alice Updated', 'new@example.com', '2026-01-01 11:00:00');
```

```
-- Дубли видны!
SELECT * FROM users_replacing WHERE user_id = 1;
-- -> 2 строки
```

1.1.11.1. Способы получить дедуплицированные данные

1. **FINAL** — дедупликация «на лету» при чтении:

```
SELECT * FROM users_replacing FINAL;
```

2. **argMax** — ручная дедупликация через агрегацию:

```
SELECT
    user_id,
    argMax(name, updated_at) AS name,
    argMax(email, updated_at) AS email,
    max(updated_at) AS updated_at
FROM users_replacing
GROUP BY user_id;
```

3. **OPTIMIZE TABLE ... FINAL** — принудительный merge:

```
OPTIMIZE TABLE users_replacing FINAL;
-- После этого дубли физически удалены
```

4. **Materialized View** — дедупликация при вставке в отдельную таблицу.

1.2. Лекция 2: RBAC и ABAC: модели управления доступом в ИБ

В этой лекции рассматриваются базовые модели авторизации как независимый слой архитектуры безопасности. Фокус — не на конкретной СУБД, а на том, как формально и операционно проектировать управление доступом в корпоративных системах.

1.2.1. Зачем нужны модели управления доступом

Аутентификация отвечает на вопрос «кто это?», а авторизация — «что этому субъекту разрешено делать?». Ошибки именно в авторизации часто приводят к критическим инцидентам: утечке данных, эскалации привилегий, нарушению принципа минимально необходимых прав.

Любая модель авторизации оперирует тремя базовыми сущностями:

- **Субъект** — кто запрашивает доступ: пользователь, сервис, процесс.
- **Объект** — к чему запрашивается доступ: таблица, файл, API-метод, запись.
- **Действие** — что именно субъект хочет сделать: чтение, запись, удаление, администрирование.

Модели отличаются тем, **как** они связывают субъекта с разрешёнными действиями над объектами. RBAC делает это через роли, ABAC — через атрибуты и контекст запроса.

1.2.2. RBAC: роль как единица администрирования

RBAC (Role-Based Access Control) строится на идее, что права назначаются не напрямую пользователям, а ролям. Пользователь получает права через принадлежность к роли.

Сущность RBAC	Назначение
Пользователь	Человеческий или машинный субъект, выполняющий действия
Роль	Функция в организации: аналитик, администратор, оператор
Привилегия	Разрешённая операция над объектом
Сеанс	Активированный поднабор ролей пользователя в рамках одного подключения. Пользователь может иметь несколько ролей, но в конкретной сессии работать только с частью из них — это ограничивает радиус возможного ущерба

Таблица 3. Базовые сущности RBAC

Преимущество RBAC — управляемость: при кадровых изменениях обычно достаточно изменить состав ролей, не пересобирая матрицу прав для каждого пользователя.

1.2.3. Иерархии ролей и разделение обязанностей

В зрелых системах роли образуют иерархию: старшая роль наследует права младшей. Это снижает дублирование политик и упрощает сопровождение.

Помимо иерархии, RBAC позволяет вводить ограничения на совмещение ролей. Классический пример: один сотрудник не должен иметь возможность и инициировать платёж, и самому же его утверждать — это открывает путь к мошенничеству без внешнего контроля. Такой принцип называется SoD (Separation of Duties, разделение обязанностей) и реализуется двумя способами:

- **Статическое SoD** — пользователь не может одновременно состоять в конфликтующих ролях. Система просто не позволит назначить обе роли одному человеку.
- **Динамическое SoD** — пользователь может иметь обе роли, но не может активировать их в рамках одной сессии или одной бизнес-операции.

1.2.4. ABAC: политика на основе атрибутов

RBAC отвечает на вопрос «к какой роли принадлежит пользователь?» и на основании этого разрешает или запрещает действие. Но иногда этого недостаточно: два аналитика имеют одинаковую роль, однако один работает из офиса, другой — из кафе за рубежом. Должны ли они иметь одинаковый доступ к чувствительным данным?

ABAC (Attribute-Based Access Control) решает эту задачу, добавляя к роли контекст запроса. Решение о доступе принимается на основе атрибутов:

- **Кто:** отдел, должность, уровень допуска.
- **Что:** тип данных, владелец ресурса, метка конфиденциальности.
- **Как:** чтение, запись, экспорт.
- **Когда и откуда:** рабочее время, корпоративная сеть, тип устройства.

Пример политики: «разрешить чтение отчётов отдела продаж, если сотрудник из того же отдела, подключается из офисной сети и сейчас рабочий день». Аналитик из другого отдела или тот же сотрудник из домашней сети — получают отказ, хотя роль у них одна и та же.

1.2.5. Как работает проверка доступа в ABAC

Когда пользователь обращается к ресурсу, система проходит четыре шага:

1. **Запрос перехватывается** в точке применения политики (PEP) — это может быть API-шлюз, прокси или middleware в приложении.
2. **Собираются атрибуты:** система запрашивает данные о пользователе из HR-системы, об устройстве — из MDM, о времени — из системных часов (PIP).
3. **Принимается решение:** движок политик (PDP) проверяет атрибуты против набора правил — тех самых, что администратор безопасности описал заранее — и выдаёт «разрешить» или «запретить».
4. **Решение применяется:** тот же PEP, что перехватил запрос, пропускает его или блокирует.

Такое разделение позволяет отдельно менять политики, отдельно — источники атрибутов, и при этом иметь полный аудитный след: почему конкретный запрос был разрешён или отклонён.

1.2.6. RBAC и ABAC: сравнение

Теперь, когда обе модели понятны, можно сравнить их по ключевым критериям.

Критерий	RBAC	ABAC
Основная единица	Роль	Атрибут + правило
Сложность внедрения	Ниже	Выше
Гибкость	Средняя	Высокая
Прозрачность для аудита	Высокая при корректной роли-модели	Зависит от качества политик и атрибутов
Типичный риск	Взрыв ролей: со временем ролей становится так много, что матрица прав теряет управляемость	Взрыв политик и рассогласованные атрибуты: правила множатся и начинают конфликтовать
Когда уместно	Стабильные бизнес-функции	Динамичный контекст доступа

Таблица 4. Сравнение RBAC и ABAC

1.2.7. Комбинированный подход RBAC + ABAC

На практике обе модели применяются вместе, потому что каждая закрывает слабость другой: RBAC прост и прозрачен, но не учитывает контекст; ABAC гибок, но сложен в администрировании.

Типичная гибридная схема: RBAC задаёт грубые границы полномочий, ABAC накладывает контекстные ограничения поверх роли. Например, роль «аналитик» даёт право читать витрины данных, а ABAC-политика уточняет: только своего домена, только из корпоративной сети, только в рабочее время.

1.2.8. Типичные ошибки проектирования доступа

- Назначение прав напрямую пользователям в обход ролевой модели — это ломает управляемость.
- Избыточные «универсальные» роли с административными правами вместо гранулярных.
- Отсутствие периодической ревизии: роли и атрибуты устаревают вместе с оргструктурой.
- Смешение бизнес-ролей (аналитик, менеджер) и технических ролей (read-only, admin) без явной границы.
- Отсутствие аудитного следа: если нельзя объяснить, почему запрос был разрешён, модель не считается зрелой.

1.3. Лекция 3: Распределённый ClickHouse: шарды, реплики и координация

Эта лекция фокусируется на том, как ClickHouse масштабируется за пределы одного узла. Цель — связать логические понятия кластера (шард, реплика, distributed-таблица) с практическими эффектами: пропускная способность, отказоустойчивость, сложность эксплуатации.

1.3.1. Координация: ZooKeeper и ClickHouse Keeper

ZooKeeper используется как координационная система в распределённых кластерах ClickHouse: репликация, распределённые DDL, синхронизация состояния. ZooKeeper — отдельный сервис на Java.

ClickHouse Keeper — более новая альтернатива, написанная на C++ и оптимизированная под ClickHouse. При прочих равных Keeper снижает операционную сложность кластера, поскольку использует единый технологический стек и лучше интегрирован с механизмами самой СУБД.

1.3.2. Шарды и реплики

ClickHouse для распределения данных использует два ключевых понятия.

Шард (shard) — часть общего набора данных. Каждый шард хранит собственный фрагмент таблицы. Например, при диапазонном разбиении логов за 12 месяцев:

- Шард 1: январь–апрель,
- Шард 2: май–август,
- Шард 3: сентябрь–декабрь.

Реплика (replica) — копия шарда для отказоустойчивости и балансировки чтения.

	Реплика 1	Реплика 2
Шард 1	Сервер A	Сервер B
Шард 2	Сервер C	Сервер D
Шард 3	Сервер E	Сервер F

Таблица 5. Пример топологии: 3 шарда x 2 реплики

Итоговое разделение ответственности:

- **Шарды** — горизонтальное масштабирование по данным и нагрузке.
- **Реплики** — отказоустойчивость и масштабирование чтения.

1.3.3. Локальные и распределённые таблицы

В кластере обычно используются два уровня таблиц.

1. **Локальные таблицы** (ReplicatedMergeTree) существуют на каждом узле и реально хранят данные:

```
CREATE TABLE events_local ON CLUSTER my_cluster (  
    id UInt64,  
    ts DateTime,  
    user_id UInt32  
) ENGINE = ReplicatedMergeTree(  
    '/clickhouse/tables/{shard}/events_local',  
    '{replica}'  
)  
PARTITION BY toYYYYMM(ts)  
ORDER BY (ts, user_id);
```

2. **Распределённая таблица** (Distributed) хранит только метаданные маршрутизации и проксирует запросы на локальные таблицы:

```
CREATE TABLE events_all ON CLUSTER my_cluster AS events_local  
ENGINE = Distributed(my_cluster, default, events_local, user_id);
```

1.3.4. Путь записи и чтения

При проектировании важно понимать, где именно выполняется операция:

- INSERT в локальную таблицу записывает данные сразу на конкретный шард.

- INSERT в Distributed-таблицу маршрутизирует строки по шару согласно sharding_key.
- SELECT из Distributed рассылает подзапросы по шардам, после чего инициатор агрегирует результат.

По этой причине стоимость запроса определяется не только сложностью SQL, но и сетевой топологией, равномерностью шардирования и объёмом межузловой передачи данных.

1.3.5. Операционные риски распределённого кластера

- **Перекося данных** при неудачном sharding_key: часть шардов перегружена, часть простаивает.
- **Сетевые задержки** увеличивают время отклика распределённых агрегирующих запросов.
- **Долгие merge/мутации** на одной реплике могут вызывать рассогласование производительности между репликами.
- **Сложность DDL-изменений** возрастает: схемы и настройки должны применяться синхронно по всему кластеру.

1.3.6. Практические ориентиры проектирования

- Выбирать sharding_key, который распределяет данные равномерно и совпадает с основными паттернами запросов.
- Хранить «горячие» данные компактно, чтобы минимизировать межшардовые сканы.
- Использовать ON CLUSTER для схемной консистентности объектов.
- Планировать ёмкость с учётом не только объёма хранения, но и сетевого трафика между узлами.

С инженерной точки зрения распределённый ClickHouse — это баланс между скоростью аналитики, стоимостью инфраструктуры и управляемой сложностью эксплуатации.

1.4. Лекция 4: Обезличивание персональных данных: модели и техники

Эта лекция формирует теоретическую базу для лабораторной работы 2. Фокус — на формальных требованиях к обезличиванию, основных техниках трансформации данных и регуляторном контексте, определяющем, когда данные считаются действительно обезличенными.

1.4.1. Регуляторный контекст

Персональные данные (ПДн) — любая информация, позволяющая прямо или косвенно идентифицировать физическое лицо. Регуляторы разграничивают два понятия:

- **Псевдонимизация** — замена идентификаторов на псевдонимы; обратимая при наличии ключа или таблицы соответствия. Данные остаются персональными по GDPR / 152-ФЗ.
- **Анонимизация** — необратимое преобразование, после которого идентификация субъекта невозможна даже при наличии дополнительных источников. Только такие данные выходят из-под действия законодательства о ПДн.

На практике граница размыта: данные, кажущиеся анонимными, могут быть деанонимизированы через связку с внешними датасетами (**атака реидентификации**).

1.4.2. Квази-идентификаторы

Прямые идентификаторы — поля, однозначно указывающие на человека: ФИО, email, СНИЛС, паспортные данные.

Квази-идентификаторы — поля, которые по отдельности безвредны, но в комбинации восстанавливают личность: дата рождения + пол + почтовый индекс. Исследования показывают, что такая тройка идентифицирует более 85% населения США.

Корректное обезличивание должно устранять угрозу реидентификации не только через прямые идентификаторы, но и через комбинации квази-идентификаторов.

1.4.3. k-анонимность

k-анонимность — формальная модель: каждая запись в датасете неотличима от не менее k–1 других записей по всем квази-идентификаторам. При k=1 каждая запись уникальна — полная уязвимость; при k=10 злоумышленник не может сузить пул кандидатов ниже 10 человек.

Достигается через:

- **Обобщение (generalization)** — замена точного значения диапазоном или категорией (возраст 27 → «20–29», город → регион).
- **Подавление (suppression)** — удаление строк или значений, нарушающих порог k.

k-анонимность не является достаточной гарантией, если чувствительный атрибут однороден внутри группы (атака по однородности). Более сильные модели (l-diversity, t-closeness) устраняют этот изъян, но для большинства практических задач курса k-анонимность достаточна.

1.4.4. Основные техники обезличивания

Техника	Описание	Обратимость
Маскирование	Замена символов: +7 921 ***-**-34	Нет (для замаскированных частей)
Хеширование	SHA-256 с солью; email → хеш	Нет (без соли/радужных таблиц)
Обобщение	Точное значение → диапазон/категория	Нет
Псевдонимизация	Замена на синтетический ID; ключ хранится отдельно	Да (с ключом)
За шумление	Добавление случайного сдвига к числовым полям	Нет

Таблица 6. Техники обезличивания и их свойства

Выбор техники зависит от цели: если данные нужны для аналитики (средний чек по возрастным группам), обобщение сохраняет статистическую ценность. Если данные используются как ключ в пайплайне (связка событий одного пользователя), подходит хеширование с солью.

1.4.5. Обезличивание в OLAP-контексте

В колоночных аналитических системах нет UPDATE в OLTP-смысле — мутации медленные и оставляют данные в неконсистентном состоянии во время выполнения. Это означает, что обезличивание в ClickHouse — это **ETL-задача**: чтение исходных данных, трансформация в отдельном пайплайне (в SQL внутри ClickHouse или внешним инструментом), запись в новую таблицу. Исходная таблица с ПДн после верификации удаляется.

Такой подход одновременно соответствует архитектурному паттерну (иммутабельность данных) и регуляторному требованию (возможность подтвердить факт уничтожения оригинальных ПДн).

1.5. Лекция 5: Materialized Views: инкрементальная агрегация

Эта лекция объясняет механизм Materialized Views в ClickHouse: как он устроен, чем отличается от представлений в OLTP-системах, для каких задач подходит и где его применение некорректно. Цель — сформировать у студента архитектурное понимание, достаточное для осознанного проектирования слоя производных данных поверх сырых логов.

1.5.1. VIEW, Materialized View и Table — три разных объекта

Прежде чем разбирать механизм MV в ClickHouse, важно чётко разделить три понятия, которые часто путают.

VIEW (обычное представление) — это не таблица и не данные. Это *сохранённый кусок SQL-запроса*. При каждом обращении к VIEW база данных подставляет его определение в запрос и выполняет полный пересчёт. VIEW не хранит ни одной строки — это просто удобный алиас для сложного SELECT.

TABLE (таблица) — физически хранит данные. Данные попадают туда через INSERT и лежат на диске. Чтение таблицы — это чтение реальных данных, без пересчёта.

Materialized View (материализованное представление) — нечто среднее, и именно здесь начинается путаница. MV *материализует* (т.е. физически сохраняет) результат запроса — но **механизм обновления различается от СУБД к СУБД**:

- **PostgreSQL**: MV — это *снимок* результата запроса. Данные вычисляются один раз при создании и хранятся как таблица. Чтобы обновить, нужно **вручную** выполнить REFRESH MATERIALIZED VIEW. До вызова REFRESH данные устаревают.
- **Oracle**: MV может обновляться по расписанию (ON DEMAND), при коммите (ON COMMIT), или через механизм fast refresh (инкрементально по логам изменений).
- **ClickHouse**: MV — это **триггер на INSERT**. При каждой вставке в исходную таблицу ClickHouse применяет SELECT-запрос из определения MV к *только что вставленному блоку* и записывает результат в отдельную физическую таблицу. Ручного REFRESH нет — обновление происходит автоматически и инкрементально.

Свойство	VIEW	TABLE	MV (PostgreSQL)	MV (ClickHouse)
Хранит данные?	Нет — пересчёт	Да	Да (снимок)	Да (в целевой таблице)
Когда обновляется	При каждом SELECT	При INSERT/UPDATE/DELETE	Вручную (REFRESH)	Автоматически при INSERT
Стоимость чтения	Полный пересчёт	Чтение с диска	Чтение с диска	Чтение готового агрегата
Стоимость записи	Нулевая	Запись на диск	Нулевая (до REFRESH)	Доп. запись в целевую таблицу

Таблица 7. VIEW vs Table vs Materialized View

1.5.2. Главное правило: разберитесь, как работает *ваши* MV

Слово «Materialized View» в разных СУБД означает разные механизмы. Если вы встретили MV в рабочем проекте — **не поленитесь выяснить**:

- **Когда он пересчитывается?** Автоматически при вставке (ClickHouse)? Вручную по команде REFRESH (PostgreSQL)? По расписанию (Oracle)?
- **Как он пополняется?** Полный пересчёт с нуля или инкрементально?
- **Что будет, если данные в источнике изменятся?** В ClickHouse целевая таблица MV — независимый объект, удаление из source её не затрагивает. В PostgreSQL REFRESH пересчитает всё заново.

Не зная ответов на эти вопросы, легко получить устаревшие данные, ложные алерты или пропущенные инциденты.

1.5.3. Архитектура MV в ClickHouse: три объекта

Materialized View в ClickHouse — это связка трёх объектов:

- 1. **Исходная таблица** — таблица, в которую поступают данные. MV срабатывает на каждый INSERT в эту таблицу.
- 2. **Целевая таблица (target table)** — обычная таблица ClickHouse, куда MV записывает результат. У неё свой движок, свой ORDER BY, свой TTL. Именно к ней обращаются аналитические запросы.
- 3. **Materialized View** — объект, связывающий первые два. Содержит SELECT-запрос, который применяется к каждому вставленному блоку.

```
-- 1. Исходная таблица
CREATE TABLE security_events (
    timestamp DateTime,
    source_ip String,
    event_type String,
    status String
) ENGINE = MergeTree()
ORDER BY (event_type, source_ip, timestamp);

-- 2. Целевая таблица
CREATE TABLE failed_logins_per_minute (
    minute DateTime,
    source_ip String,
    attempts UInt64
) ENGINE = SummingMergeTree()
ORDER BY (source_ip, minute);

-- 3. Materialized View
CREATE MATERIALIZED VIEW mv_failed_logins
TO failed_logins_per_minute
AS SELECT
    toStartOfMinute(timestamp) AS minute,
    source_ip,
    count() AS attempts
FROM security_events
WHERE status = 'failed' AND event_type = 'login'
GROUP BY minute, source_ip;
```

Ключевой момент: MV **не** хранит данные сама. Она только описывает трансформацию. Данные хранит целевая таблица — и к ней можно обращаться напрямую, как к любой другой таблице.

1.5.4. Выбор движка целевой таблицы

Движок целевой таблицы определяет, как агрегаты будут объединяться при merge:

Движок	Поведение при merge	Типичное применение
MergeTree	Строки просто накапливаются	Детализированные проекции без агрегации
SummingMergeTree	Суммирует числовые колонки по ключу	Счётчики, суммы
AggregatingMergeTree	Объединяет агрегатные состояния	Сложные агрегаты: uniq, quantile, avg

Таблица 8. Движки целевых таблиц для MV

SummingMergeTree подходит, когда агрегация сводится к count() или sum(). Если нужны uniqExact, quantile или avg, требуется AggregatingMergeTree с функциями агрегатных состояний (-State / -Merge).

1.5.5. Как MV обрабатывает данные

Важно понимать, что MV видит **только текущий вставляемый блок**, а не всю таблицу. Это означает:

- GROUP BY в MV группирует строки **внутри блока**, а не по всей истории.

- Если два INSERT содержат события с одним и тем же ключом (`source_ip`, `minute`), в целевой таблице появятся **две строки** с этим ключом.
- Объединение этих строк произойдёт позже, при фоновом `merge` (или при чтении с `FINAL`).

Именно поэтому для целевой таблицы обычно выбирают `SummingMergeTree` или `AggregatingMergeTree` — они корректно объединяют частичные агрегаты при `merge`.

1.5.6. Когда Materialized View подходит

MV эффективна, когда выполняются три условия одновременно:

1. **Правило агрегации фиксировано и известно заранее.** Ключ группировки и функция агрегации не меняются от запроса к запросу.
2. **Результат инкрементален.** Новые вставки добавляют к агрегату, не требуя пересчёта истории.
3. **Агрегат нужен часто.** Один и тот же производный показатель запрашивается многократно — в дашборде, алерте, оперативном отчёте.

Примеры подходящих задач:

- количество неудачных логинов по IP за минуту;
- суммарный трафик по серверу за час;
- число ошибок по типу за день.

1.5.7. Когда Materialized View не подходит

Не всякая аналитика может быть корректно выражена как инкрементальное преобразование потока вставок. Ниже — три класса задач, для которых MV либо не подходит, либо требует принципиально другого подхода.

1.5.7.1. Динамическое ранжирование (top-N)

«Топ-5 IP с наибольшим числом неудачных логинов за всё время»

MV умеет поддерживать счётчик по ключу, но не умеет поддерживать **глобальный рейтинг**. Каждая новая вставка может изменить порядок в итоговом списке. Рейтинг — это свойство **всего датасета**, а не отдельного блока.

Правильный подход: хранить счётчики в MV, а ранжирование выполнять запросом к целевой таблице по требованию.

1.5.7.2. Сравнение с историческим базовым уровнем

«Пользователь подозрителен, если число отказов за сегодня превышает его среднее за 30 дней более чем в 3 раза»

Здесь результат зависит не только от текущего блока, но и от исторических данных. Среднее за 30 дней меняется каждый день. MV не имеет доступа к уже накопленным данным — она видит только текущую вставку.

Правильный подход: MV поддерживает дневные счётчики, а сравнение с базовым уровнем выполняется отдельным запросом или регулярным ETL-процессом.

1.5.7.3. Оконная и позиционная логика

«Найти третью неудачную попытку логина после последнего успешного входа»

Такая задача требует упорядочения событий по времени и анализа последовательности. MV не может «помнить» предыдущие события — она обрабатывает каждый блок изолированно.

Правильный подход: запрос к сырой таблице с оконными функциями.

1.5.8. Сырые данные и агрегаты: разные роли

Одна из ключевых архитектурных идей, связанных с MV:

Аспект	Сырая таблица	Целевая таблица MV
Назначение	Расследование, ретроспектива, ad-hoc анализ	Дешёвые повторяющиеся оперативные запросы
Полнота	Все поля, все события	Только агрегированные показатели
Стоимость запроса	Высокая (полное сканирование)	Низкая (готовый агрегат)
Гибкость	Любой запрос	Только заранее определённые агрегаты

Таблица 9. Роли сырых данных и предагрегатов

MV **не заменяет** сырые данные. Она добавляет поверх них слой производных данных для конкретных, заранее известных, часто повторяющихся запросов.

Это знание переносится за пределы ClickHouse: разделение на слой сырых данных и агрегатный слой — стандартный паттерн в наблюдаемости, продуктовой аналитике и антифрод-системах.

1.5.9. Операционные особенности

- **Колонки сопоставляются по имени, не по позиции.** ClickHouse сопоставляет колонки SELECT-запроса MV с колонками целевой таблицы по именам. Если имя не совпадает — колонка получит значение по умолчанию, а не ошибку. Поэтому в SELECT всегда стоит использовать явные алиасы (AS minute, AS attempts).
- **Ошибка в MV может уронить INSERT.** По умолчанию, если MV не может записать в целевую таблицу, весь INSERT в исходную таблицу завершается ошибкой. Настройка `materialized_views_ignore_errors = 1` меняет это поведение, но скрывает проблемы.
- **POPULATE опасен.** Синтаксис `CREATE MATERIALIZED VIEW ... POPULATE` заполняет целевую таблицу историческими данными, но строки, вставленные **во время** выполнения `POPULATE`, будут потеряны. Безопаснее создать MV без `POPULATE`, а затем заполнить целевую таблицу вручную через `INSERT INTO ... SELECT`.

1.5.10. Типичные ошибки при работе с MV

- **«MV — это кэш».** Нет. Кэш можно инвалидировать и пересчитать. MV пишет результат при вставке — если вставка уже произошла, результат в целевой таблице не пересчитается.
- **«Удаление из исходной таблицы удалит агрегат».** Нет. Целевая таблица — независимый объект. Удаление строк из исходной таблицы не затрагивает уже записанные агрегаты.
- **«MV ускоряет любой запрос».** Нет. MV ускоряет только те запросы, которые совпадают с её определением. Для ad-hoc анализа нужна сырая таблица.
- **«Если аналитика важная, её надо завернуть в MV».** Нет. MV подходит только для инкрементальных агрегатов. Часть важной аналитики требует полного пересчёта или оконных функций.

1.6. Лекция 6: Private Set Intersection: идея, модель угроз и упрощённый DH-протокол

Private Set Intersection (PSI) — протокол, позволяющий двум сторонам узнать пересечение своих наборов, не раскрывая друг другу остальные элементы. В контексте курса он важен как пример архитектурного решения для задач с чувствительными данными.

1.6.1. Зачем нужен PSI

Во многих задачах информационной безопасности и аналитики нескольким организациям нужно сопоставить данные, не объединяя их в одну открытую таблицу. Типичный пример — банк и маркетплейс хотят понять, сколько у них общих клиентов, чтобы оценить риск-модель или эффективность совместной кампании, но не готовы передавать друг другу полные клиентские базы.

Обычный INNER JOIN решает вычислительную задачу, но плохо решает задачу доверия: одна из сторон или общий оператор получает доступ ко всем значениям ключа. PSI нужен в более узкой постановке: когда требуется узнать **только пересечение** и не раскрывать элементы вне него.

1.6.2. Почему обычного хеширования недостаточно

Первый интуитивный подход — захешировать email и сравнить хеши. Это лучше, чем обмениваться открытыми значениями, но не устраняет основную проблему.

- Если обе стороны используют один и тот же детерминированный хеш без секрета, они всё равно могут сравнить **все** элементы своих множеств.
- Для email и других значений из относительно предсказуемого пространства возможен перебор словаря: злоумышленник заранее считает хеши популярных адресов и ищет совпадения.
- Даже соль не решает задачу полностью, если соль известна обеим сторонам: сравнение всех элементов по-прежнему возможно.

Иначе говоря, простое хеширование скрывает исходную строку от поверхностного наблюдения, но не ограничивает сам **объём раскрытия**. PSI интересен именно тем, что меняет модель раскрытия, а не только форму представления данных.

1.6.3. Модель угроз: что защищает PSI, а что нет

В учебной постановке будем считать, что стороны **следуют протоколу**, но пытаются извлечь из полученных сообщений как можно больше информации. Такая модель называется «честный, но любопытный» участник (**semi-honest** в терминологии протоколов).

В этой модели PSI обычно даёт следующие свойства:

- сторона не узнаёт полный набор другой стороны;
- вне пересечения элементы не раскрываются в открытом виде;
- итог вычисления сводится к факту совпадения элементов.

При этом часть информации всё равно может утекать:

- размер наборов часто известен из объёма переданных сообщений;
- размер пересечения обычно узнаётся одной или обеими сторонами;
- протокол не защищает от некорректной нормализации данных: Alice@example.com и alice@example.com без приведения к каноническому виду будут считаться разными значениями;
- защита от активно нарушающего протокол участника требует более сложных схем и дополнительных доказательств корректности.

1.6.4. Идея PSI: узнать пересечение без раскрытия полных множеств

Концептуально PSI решает задачу равенства элементов под «ослеплением»¹. Стороны преобразуют свои элементы так, чтобы:

- одинаковые исходные значения после согласованной цепочки преобразований дали одинаковый результат;

¹**Ослепление** (blinding) — криптографический приём: значение преобразуется случайным или секретным множителем так, что исходный элемент восстановить нельзя, но вычислять над ним операции по-прежнему возможно.

- по промежуточным значениям нельзя было восстановить полный исходный набор;
- сравнение происходило уже над производными токенами, а не над открытыми идентификаторами.

Если задача сформулирована именно как «найти общих клиентов», а не «посчитать агрегаты по объединённой таблице», PSI оказывается естественным инструментом. Он уже, чем общее безопасное многопартийное вычисление, но и существенно проще.

1.6.5. Упрощённый протокол на основе Diffie-Hellman

Для учебной работы достаточно упрощённой схемы в мультипликативной группе по модулю большого простого числа p . Это не промышленная реализация, а учебная модель: в ней видна идея двойного ослепления, остальные детали опущены.

Пусть:

- у организации А есть секрет a ;
- у организации В есть секрет b ;
- функция $h(x)$ — по сути хэш-функция, но с прицельным выходом: вместо произвольной битовой строки она возвращает целое число из $[1, p-1]$, то есть элемент нашей группы. В учебной версии достаточно $\text{SHA-256}(x) \bmod p$. Детерминизм здесь принципиален: одинаковый email должен всегда давать одно и то же число — иначе токены двух сторон никогда не совпадут.

Тогда схема выглядит так:

1. А для каждого своего email x вычисляет $h(x)^a \bmod p$ и отправляет результаты В.
2. В повторно ослепляет их своим секретом и получает $h(x)^{ab} \bmod p$.
3. В для каждого своего email y вычисляет $h(y)^b \bmod p$ и отправляет эти значения А.
4. А доослепляет их своим секретом и получает $h(y)^{ab} \bmod p$.
5. А сравнивает два набора токенов $h(\cdot)^{ab}$ и находит совпадения.

Ключевая идея состоит в том, что обе стороны приходят к одному и тому же итоговому представлению для одинакового элемента, но ни одна из них не раскрывает второй стороне исходный набор в открытом виде.

Важно не перепутать две разные задачи: **найти совпавший токен и восстановить, какому email он соответствует у А**. Обратного преобразования $h(x)^{ab} \rightarrow x$ здесь нет. А восстанавливает соответствие не криптографически, а организационно: она изначально знает, какой email стоял на какой позиции в отправленном списке, а В при доослеплении не меняет порядок элементов.

Простой пример:

- у А список $[alice, bob, carol]$;
- после первого шага А отправляет $[h(alice)^a, h(bob)^a, h(carol)^a]$;
- В возвращает результаты в том же порядке: $[h(alice)^{ab}, h(bob)^{ab}, h(carol)^{ab}]$.

Поэтому А знает, что второй элемент ответа по-прежнему соответствует bob, хотя самого секрета b она не знает. Если затем среди токенов В находится совпадение со вторым элементом, А делает вывод, что общий email — bob. Именно сохранение порядка делает соответствие $h(x)^a \rightarrow h(x)^{ab}$ наблюдаемым для А.

1.6.6. Почему эта схема работает

Протокол опирается на коммутативность возведения в степень в группе:

- А получает для своих элементов вид $h(x)^{ab}$;
- В позволяет А получить для элементов В тот же вид $h(y)^{ab}$.

Если $x = y$, итоговые токены совпадают. Если элементы различны, совпадения не возникает, кроме пренебрежимо малой вероятности коллизии отображения h .

Практический смысл такой: сравнение переносится из пространства открытых email в пространство производных токенов. Это не делает вычисление бесплатным, но уменьшает объём раскрытия относительно прямого JOIN.

1.6.7. Архитектурная схема: ClickHouse хранит данные, Python выполняет криптографию

Для курса важно отделять роль СУБД от роли вычислительного слоя.

ClickHouse здесь удобен для:

- хранения исходных клиентских наборов;
- быстрой выборки входных данных;
- контрольного JOIN в целях верификации;
- записи результатов запуска в append-only таблицу.

Сама криптография живёт вне СУБД, например в Python-оркестраторе. Это устойчивый архитектурный подход: аналитическая база отвечает за хранение и SQL-операции, а специализированная логика безопасности и протоколов выполняется во внешнем сервисе.

В промышленных системах этот внешний слой реализуется специализированными библиотеками или фреймворками. Google выпустил **Private Join and Compute** — реализацию именно того DH-варианта, что описан выше, применяемую для атрибуции рекламных конверсий. Meta использует **Private-ID** (Rust) как стадию сопоставления внутри более крупного пайплайна безопасных многосторонних вычислений² для рекламной аналитики. Microsoft Research поддерживает **APSI** — асимметричный вариант для случаев, когда наборы сторон сильно различаются по размеру. Signal применяет PSI для поиска контактов: клиент узнаёт, кто из адресной книги зарегистрирован в мессенджере, не передавая на сервер полный список контактов. Везде прослеживается одна тенденция: PSI — не самостоятельный сервис, а стадия сопоставления перед основным вычислением.

1.6.8. Ограничения и компромиссы PSI

PSI не является «волшебной анонимизацией». Он решает конкретную задачу и расплачивается за это ограничениями:

- **Накладные расходы по процессору и сети.** Криптографические преобразования и обмен токенами почти всегда дороже прямого JOIN внутри СУБД.
- **Узкий класс задач.** PSI хорошо подходит для проверки пересечения, но не заменяет произвольные совместные вычисления.
- **Чувствительность к качеству входных данных.** Без нормализации email, удаления дублей и согласованной кодировки результат будет некорректным.
- **Утечка метаданных.** Размеры наборов и часто размер пересечения всё равно становятся известны.

Поэтому PSI уместен там, где модель доверия важнее чистой скорости. Если же обе стороны и так работают в одной доверенной среде, прямой JOIN почти всегда проще и дешевле.

²MPC (Multi-Party Computation) — класс протоколов, позволяющих нескольким сторонам совместно вычислить функцию от своих данных, не раскрывая сами данные друг другу. PSI — частный случай MPC для задачи пересечения множеств.

1.7. Лекция 7: Выбор подхода для приватного сопоставления и совместной аналитики

Архитектурный выбор в этой области определяется не технологической новизной инструмента, а конкретными требованиями: что именно нужно вычислить, кому можно доверять данные и какова цена ошибки.

1.7.1. Сначала формулируем задачу, а не выбираем инструмент

Перед выбором подхода нужно ответить на три вопроса:

1. Что именно нужно получить: полное объединение данных, только пересечение, агрегат или произвольную функцию?
2. Кому можно доверять открытые данные: обеим сторонам, общей платформе, только изолированному окружению или никому?
3. Какова цена ошибки: допустима ли утечка части метаданных, насколько критична производительность, нужен ли аудит вычисления?

Без этой постановки команды часто выбирают либо слишком слабую защиту («давайте просто захешируем»), либо чрезмерно тяжёлую технологию там, где обычного JOIN было бы достаточно.

1.7.2. Обычный JOIN: когда доверенная среда уже есть

Обычный INNER JOIN — это правильное решение, если данные уже легально и организационно могут находиться в одной доверенной среде. Например, одна компания сводит две свои внутренние системы или обе стороны заранее согласились работать через общего оператора данных.

Преимущества:

- минимальная вычислительная стоимость;
- простая реализация и отладка;
- полный язык SQL для последующей аналитики.

Ограничение: среда выполнения видит исходные значения ключей. Если это неприемлемо по модели доверия или регуляторным причинам, JOIN перестаёт подходить независимо от его скорости.

1.7.3. Обезличивание и псевдонимизация: когда достаточно трансформации данных

Если задача не требует сопоставлять данные между независимыми владельцами по исходному идентификатору, часто достаточно обезличивания или псевдонимизации.

Практический пример: компания хочет передать команде аналитики события клиентов без прямых идентификаторов. Если аналитика строится по возрастным группам, регионам и сегментам, разумнее обобщить или подавить поля, чем внедрять PSI.

Но здесь важно различать два класса задач:

- **обезличивание** уменьшает риск идентификации в уже передаваемом наборе данных;
- **PSI** позволяет вообще не передавать полный набор значений второй стороне.

Именно поэтому обезличивание не заменяет PSI в задаче «найти общих клиентов у двух организаций, не раскрывая полный список клиентов каждой».

1.7.4. PSI: когда нужен только факт пересечения

PSI уместен, когда:

- у данных разные владельцы;
- стороны не хотят раскрывать полные множества;
- итоговая задача сводится к нахождению общих элементов.

Типичные сценарии:

- поиск общих клиентов у банка и страховой компании;
- совместный антифрод между платформами;
- сверка списков скомпрометированных идентификаторов.

Главный компромисс: PSI защищает лучше, чем обычный JOIN, но и умеет меньше. Как только требуется не только пересечение, а более богатая аналитика по объединённым данным, одной схемы PSI обычно уже недостаточно.

1.7.5. TEE: когда нужен более общий вычислительный сценарий

TEE (Trusted Execution Environment, доверенная изолированная среда выполнения) — это аппаратно и системно изолированный контур, внутри которого можно выполнять код над чувствительными данными так, чтобы даже оператор инфраструктуры не видел содержимое памяти в открытом виде.

В отличие от PSI, TEE подходит не только для пересечения. Если нужно выполнить более общий алгоритм: скоринг, обучение модели, сложную агрегацию по объединённым данным, — TEE часто оказывается практичнее, чем попытка выразить всё через специализированные криптографические протоколы.

Но у TEE есть свои ограничения:

- доверие переносится на аппаратную и программную реализацию изолированного контура;
- сложнее операционная модель: удалённая проверка доверенной среды (attestation), обновления, контроль уязвимостей;
- возможны побочные каналы и ограничения по памяти и производительности.

1.7.6. Сравнение подходов

Подход	Что умеет	Кому доверяем	Что раскрывает-ся	Цена и слож-ность
Обычный JOIN	Полное объединение и любая SQL-аналитика	Общей среде выполнения	Исходные ключи и все присоединяемые строки	Минимальная стоимость, минимальная сложность
Обезличивание / псевдонимизация	Передача набора данных с пониженным риском идентификации	Получателю обезличенного набора	Часть структуры и статистики данных; возможен риск реидентификации	Низкая или средняя сложность
PSI	Только пересечение множеств	Протоколу и корректной реализации	Обычно размеры наборов и итог пересечения	Средняя или высокая сложность, криптографические накладные расходы
TEE	Более общие вычисления над чувствительными данными	Аппаратной изоляции и механизму удалённой проверки среды	Результат вычисления и часть метаданных выполнения	Высокая операционная сложность

Таблица 10. Сравнение подходов для совместной работы с чувствительными данными

Каждый метод уместен в своей модели доверия и своём классе вычислений. Универсально «лучшего» не существует.

1.7.7. Практические сценарии выбора

Рассмотрим четыре типовые постановки:

- **Одна компания, два внутренних контура данных.** Если юридически и организационно данные можно свести в одном хранилище, выбирается обычный JOIN.
- **Передача аналитикам обезличенного набора данных.** Если цель — снизить риск идентификации при последующем анализе, выбирается обезличивание.
- **Две организации ищут только общих клиентов.** Если полный обмен клиентскими базами недопустим, уместен PSI.

- **Две организации хотят считать общую скоринговую модель по объединённым признакам.** Если пересечения недостаточно и нужен более общий алгоритм, стоит рассматривать TEE или более широкие методы безопасных вычислений.

1.7.8. Типичные ошибки архитектурного выбора

- **Путать скрывание значения со скрыванием факта владения набором.** Хеширование может скрыть строку, но не решает задачу ограничения раскрытия так, как PSI.
- **Использовать PSI там, где нужна полноценная совместная аналитика по объединённым данным.** В этом случае протокол оказывается слишком узким.
- **Выбирать TEE «на всякий случай».** Это технологически тяжёлый путь, который оправдан только если действительно нужна общая защищённая среда вычисления.
- **Игнорировать организационную модель доверия.** Если стороны и так могут законно работать через общего оператора, криптографический протокол может быть неоправданным усложнением.

С инженерной точки зрения хороший выбор — это не максимальная криптографическая «крутость», а минимально достаточный механизм, покрывающий конкретную модель угроз, требования по производительности и регуляторные ограничения.

2. Лабораторный практикум

2.1. Введение в лабораторный практикум

Практикум состоит из четырёх лабораторных работ на стыке аналитических баз данных и информационной безопасности. Лабораторные опираются на лекцию 1 (основы ClickHouse), лекцию 2 (модели управления доступом), лекцию 3 (распределённая архитектура ClickHouse), лекцию 4 (обезличивание персональных данных), лекцию 5 (Materialized Views), лекцию 6 (PSI) и лекцию 7 (выбор подхода для приватного сопоставления и совместной аналитики).

Ключевая идея, проходящая через все лабораторные: ClickHouse — это колоночная аналитическая СУБД, а не OLTP-система. Каждая работа подсвечивает конкретное следствие этой архитектуры для задач ИБ:

- **Лабораторная 1 (RBAC):** роли, Row Policies, квоты; мутации тяжёлые, поэтому разграничение прав на ALTER критически важно.
- **Лабораторная 2 (обезличивание):** нельзя просто UPDATE ПДн, нужен ETL-пайплайн; студент сталкивается с ReplacingMergeTree и видит ограничения UPSERT-подхода для задач гарантированного уничтожения ПДн.
- **Лабораторная 3 (аудит):** Materialized View для агрегации security-событий; append-only природа — **фича** для логов безопасности, удалить следы сложно.
- **Лабораторная 4 (PSI):** иммутабельность данных усиливает изоляцию между участниками протокола, а сама работа позволяет сравнить прямой JOIN, обезличивание, PSI и более общую идею доверенной изолированной среды выполнения.

2.2. Лабораторная работа 1: RBAC в ClickHouse

2.2.1. Цель

Развернуть ClickHouse (Docker), создать таблицу с тестовыми данными, настроить ролевую модель доступа: пользователи, роли, привилегии, Row Policies, квоты. Убедиться, что разграничение прав в OLAP-системе работает иначе, чем в OLTP.

2.2.2. Подготовка

1. Развернуть ClickHouse через Docker Compose.
2. Подключиться к серверу через clickhouse-client.
3. Создать таблицу events и наполнить тестовыми данными:

```
CREATE TABLE events (  
    event_id UInt32,  
    timestamp DateTime,  
    user_id UInt32,  
    department String,  
    event_type String,  
    status String  
) ENGINE = MergeTree()  
PARTITION BY toYYYYMM(timestamp)  
ORDER BY (department, event_type, timestamp);  
  
INSERT INTO events  
SELECT  
    number,  
    now() - randUniform(0, 90*24*3600),  
    rand() % 1000,  
    ['sales','engineering','support'][rand() % 3 + 1],  
    ['login','logout','access','modify','delete']  
        [rand() % 5 + 1],  
    ['ok','error','denied'][rand() % 3 + 1]  
FROM numbers(50000);
```

2.2.3. Часть 1: Роли и привилегии

2.2.3.1. Задание

1. Создать три роли с разным уровнем доступа:

```
CREATE ROLE analyst;  
CREATE ROLE ingester;  
CREATE ROLE admin;  
  
-- analyst: только чтение  
GRANT SELECT ON default.events TO analyst;  
  
-- ingester: только вставка  
GRANT INSERT ON default.events TO ingester;  
  
-- admin: полный доступ (включая ALTER = мутации)  
GRANT ALL ON default.* TO admin;
```

2. Создать пользователей и назначить роли:

```
CREATE USER viewer IDENTIFIED BY 'viewer123'  
    DEFAULT ROLE analyst;  
CREATE USER loader IDENTIFIED BY 'loader123'  
    DEFAULT ROLE ingester;  
CREATE USER superuser IDENTIFIED BY 'admin123'  
    DEFAULT ROLE admin;
```

3. Проверить привилегии — подключиться как viewer:

```
clickhouse-client --user viewer --password viewer123
```

- `SELECT count() FROM events` — работает.
- `INSERT INTO events VALUES (...)` — ошибка `Not enough privileges`.
- `ALTER TABLE events DELETE WHERE 1` — ошибка `Not enough privileges`.

2.2.3.2. Контрольные вопросы

- Чем `GRANT ALL` отличается от перечисления `GRANT SELECT, INSERT, ALTER`?
- Что произойдёт, если пользователю назначены две роли с разными наборами прав?

2.2.4. Часть 2: Row Policies

2.2.4.1. Задание

1. Создать Row Policy — `viewer` видит только данные подразделения `sales`:

```
CREATE ROW POLICY sales_only ON events
  FOR SELECT USING department = 'sales'
  TO viewer;
```

2. Подключиться как `viewer` и выполнить:

```
SELECT department, count() FROM events GROUP BY department;
```

Результат: только строки `sales`. Остальные подразделения не видны.

3. Добавить вторую политику для другого пользователя:

```
CREATE USER eng_viewer IDENTIFIED BY 'eng123'
  DEFAULT ROLE analyst;
```

```
CREATE ROW POLICY eng_only ON events
  FOR SELECT USING department = 'engineering'
  TO eng_viewer;
```

4. Убедиться, что `eng_viewer` видит только `engineering`, а `viewer` — только `sales`.

2.2.4.2. Контрольные вопросы

- Чем Row Policy в ClickHouse отличается от `VIEW` с фильтром?
- Можно ли обойти Row Policy через подзапрос или `JOIN`?

2.2.5. Часть 3: Квоты и ограничения

2.2.5.1. Задание

1. Настроить квоту для роли `analyst`:

```
CREATE QUOTA analyst_quota_queries
  FOR INTERVAL 1 MINUTE MAX queries = 10
  TO analyst;
```

```
CREATE QUOTA analyst_quota_rows
  FOR INTERVAL 1 HOUR MAX result_rows = 100000
  TO analyst;
```

2. Подключиться как `viewer` и выполнить 11 запросов подряд — на 11-м получить ошибку `Quota exceeded`.
3. Настроить ограничение на уровне профиля:

```
CREATE SETTINGS PROFILE analyst_profile
  SETTINGS max_execution_time = 5,
           max_memory_usage = 100000000
  TO analyst;
```

4. Выполнить тяжёлый запрос от имени `viewer` и убедиться, что он прерывается по таймауту.

2.2.5.2. Контрольные вопросы

- Зачем нужны квоты в аналитической системе, если запросы только на чтение?
- Чем квоты отличаются от настроек профиля (`SETTINGS PROFILE`)?

2.2.6. Часть 4: Проверка модели доступа

2.2.6.1. Задание

1. Заполнить таблицу, подключаясь от имени каждого пользователя и проверяя каждую операцию:

Операция	viewer	loader	superuser
SELECT * FROM events	?	?	?
SELECT (видит все department?)	?	?	?
INSERT INTO events VALUES (...)	?	?	?
DROP TABLE events	?	?	?
11-й запрос за минуту (квота)	?	—	—

Таблица 11. Матрица доступа

1. Убедиться, что `system.query_log` фиксирует все действия всех пользователей, включая отклонённые запросы:

```
SELECT user, query, type, exception
FROM system.query_log
WHERE event_time > now() - INTERVAL 10 MINUTE
ORDER BY event_time DESC
LIMIT 20;
```

2.2.6.2. Вывод

RBAC в ClickHouse позволяет гранулярно разграничить доступ: от ролей и привилегий до фильтрации строк (Row Policies) и ограничения нагрузки (квоты). Все действия пользователей — включая отказы — аудируются через `system.query_log`.

2.3. Лабораторная работа 2: Обезличивание персональных данных

2.3.1. Цель

Разобрать три варианта обезличивания ПДн в ClickHouse средствами SQL: мутации, ReplacingMergeTree и ETL-пайплайн. Для каждого варианта — понять, в каких сценариях он уместен и где его применять нельзя. В финале — проверить результат обезличивания через метрику k-анонимности из лекции 4.

2.3.2. Подготовка данных

Данные генерируются один раз в таблицу-источник тремя батчами — по одному на каждый вариант обезличивания. Колонка batch используется как ключ партиционирования, что позволяет переместить каждый батч в отдельную таблицу одной командой MOVE PARTITION.

MOVE PARTITION — это операция на уровне метаданных файловой системы: данные физически не копируются, только переназначается владелец директории с партом. Поэтому она выполняется мгновенно независимо от объёма данных.

```
CREATE TABLE raw_source (  
    batch      UInt8,  
    user_id    UInt32,  
    full_name  String,  
    email      String,  
    phone      String,  
    birth_date Date,  
    city       String,  
    tx_amount  Float64,  
    version    UInt8  
) ENGINE = MergeTree()  
PARTITION BY batch  
ORDER BY user_id;  
  
-- Три идентичных батча, каждый попадёт в свой партишн.  
-- intHash32(number) даёт детерминированный хеш по номеру строки,  
-- поэтому данные одинаковы во всех трёх батчах.  
INSERT INTO raw_source  
SELECT  
    b                                     AS batch,  
    number + 1                           AS user_id,  
    concat('User_', toString(number + 1)) AS full_name,  
    concat('user', toString(number + 1), '@example.com') AS email,  
    concat('+7', toString(9000000000 + intHash32(number) % 999999999)) AS phone,  
    toDate('1970-01-01') + toIntervalDay(intHash32(number + 1) % 18000) AS birth_date,  
    ['Москва', 'Санкт-Петербург', 'Балашиха', 'Гатчина', 'Подольск']  
        [intHash32(number + 2) % 5 + 1] AS city,  
    round(intHash32(number + 3) % 100000 / 100.0, 2) AS tx_amount,  
    1                                               AS version  
FROM numbers(10000)  
CROSS JOIN (SELECT arrayJoin([1, 2, 3]) AS b);  
  
-- Целевые таблицы --- одинаковая структура, разные движки  
CREATE TABLE users_mutation AS raw_source ENGINE = MergeTree()  
    PARTITION BY batch ORDER BY user_id;  
  
CREATE TABLE users_rmt AS raw_source ENGINE = ReplacingMergeTree(version)  
    PARTITION BY batch ORDER BY user_id;  
  
CREATE TABLE users_etl AS raw_source ENGINE = MergeTree()  
    PARTITION BY batch ORDER BY user_id;  
  
-- Мгновенное перемещение партиций  
ALTER TABLE raw_source MOVE PARTITION 1 TO TABLE users_mutation;
```



```
ALTER TABLE raw_source MOVE PARTITION 2 TO TABLE users_rmt;
ALTER TABLE raw_source MOVE PARTITION 3 TO TABLE users_etl;
```

```
-- raw_source теперь пуст
SELECT count() FROM raw_source; -- 0
```

2.3.3. Вариант 1: Мутация (ALTER UPDATE)

Мутация — это механизм ClickHouse для изменения данных на месте. Подходит для разовых исправлений на небольших объёмах, когда допустима временная несогласованность и нет требований к немедленному подтверждению удаления.

2.3.3.1. Задание

1. Запустить обезличивание через мутацию:

```
ALTER TABLE users_mutation
  UPDATE email = hex(SHA256(concat('mysalt', email))),
         phone = concat('***', substring(phone, -4))
  WHERE 1
```

2. Убедиться, что мутация завершена и зафиксирована:

```
SELECT command, is_done
FROM system.mutations
WHERE table = 'users_mutation'
ORDER BY create_time DESC
LIMIT 5;
```

3. Выполнить `SELECT email FROM users_mutation LIMIT 5` — данные уже изменены, вернуть оригинал невозможно.

2.3.3.2. Контрольные вопросы

- Что вернёт `SELECT email FROM users_mutation LIMIT 5`, запущенный прямо во время мутации на большой таблице? Допустимо ли такое поведение в продакшн-системе?
- Почему отсутствие отката — проблема именно для ПДн, но не для исправления технической ошибки в числовом поле?

2.3.3.3. Когда уместно, когда нет

Уместно	Не уместно
Разовое исправление ошибки в небольшой таблице	Большие таблицы: мутация блокирует merge и нагружает диск
Изменение технических полей (не ПДн)	Нужна атомарность или возможность отката
Обезличивание, когда допустима непоследовательность данных во время выполнения	Высокая нагрузка на запись: мутация конкурирует с фоновым merge

Таблица 12. Мутации: применимость

2.3.4. Вариант 2: ReplacingMergeTree

ReplacingMergeTree предназначен для UPSERT-паттерна: вставить строку с тем же ключом и более высоким version — и при merge старая версия исчезнет. Это основной способ обновления данных в ClickHouse без мутаций.

2.3.4.1. Задание

1. Вставить обезличенные строки **в ту же таблицу** `users_rmt` с `version = 2` одним запросом:

```
INSERT INTO users_rmt
SELECT
  batch,
  user_id,
  '***' AS full_name,
  hex(SHA256(concat('mysalt', email))) AS email,
  concat('***', substring(phone, -4)) AS phone,
```

```

    birth_date,
    city,
    tx_amount,
    2
FROM users_rmt
WHERE version = 1;
AS version

```

2. Выполнить `SELECT count() FROM users_rmt` — таблица теперь содержит 20К строк: оригиналы и обезличенные копии.
3. Выполнить `SELECT email FROM users_rmt WHERE user_id = 1` — видны обе версии.
4. Выполнить `SELECT email FROM users_rmt FINAL WHERE user_id = 1` — видна только обезличенная.
5. Выполнить `OPTIMIZE TABLE users_rmt FINAL` — дождаться физического слияния.
6. После `OPTIMIZE` снова проверить `SELECT` без `FINAL` — дубли исчезли.

2.3.4.2. Контрольные вопросы

- Запустите `SELECT count() FROM users_rmt` до и после `OPTIMIZE FINAL`. Почему результаты отличаются и в какой момент ПДн физически исчезают с диска?
- Если вместо `OPTIMIZE FINAL` выполнить `ALTER TABLE users_rmt DELETE WHERE version = 1` — к какому из трёх вариантов это по смыслу ближе?

2.3.4.3. Когда уместно, когда нет

До `OPTIMIZE`: оригинальные ПДн физически лежат на диске рядом с обезличенными копиями. `FINAL` скрывает их при чтении, но не удаляет. `OPTIMIZE` — фоновая операция без гарантии времени завершения.

Уместно	Не уместно
Обновление операционных данных: изменение статуса, цены, флага	Обезличивание ПДн: нельзя подтвердить момент физического удаления
Дедупликация при параллельной вставке из нескольких источников	Требования 152-ФЗ/GDPR к гарантированному уничтожению данных

Таблица 13. ReplacingMergeTree: применимость

2.3.5. Вариант 3: ETL-пайплайн

2.3.5.1. Задание

1. Создать таблицу `anonymized_users` и наполнить её одним запросом:

```

CREATE TABLE anonymized_users (
    user_id    UInt32,
    age_group  String,
    region     String,
    email_hash String,
    tx_amount  Float64
) ENGINE = MergeTree()
ORDER BY user_id;
-- PARTITION BY не задан намеренно: колонка batch отсутствует
-- в обезличенной таблице, т.к. не несёт смысловой нагрузки.

INSERT INTO anonymized_users
SELECT
    user_id,
    concat(
        toString(intDiv(dateDiff('year', birth_date, today()), 10) * 10),
        '--',
        toString(intDiv(dateDiff('year', birth_date, today()), 10) * 10 + 9)
    )
    AS age_group,
    multiIf(
        city IN ('Москва', 'Балашиха', 'Подольск'),
        city IN ('Санкт-Петербург', 'Гатчина'),
        'Другой регион'
        'Московский регион',
        'Ленинградский регион',

```

```

    )
    hex(SHA256(concat('mysalt', email))) AS region,
    tx_amount AS email_hash,
FROM users_etl
WHERE version = 1;

```

Обратите внимание: в `anonymized_users` нет ФИО, телефона, точной даты рождения и города — только производные атрибуты.

2. После верификации удалить `users_etl` с оригинальными ПДн:

```
DROP TABLE users_etl;
```

Только после DROP факт уничтожения ПДн можно подтвердить — пока таблица существует, данные физически на диске.

3. Проверить сохранность аналитики:

```

-- Распределение по регионам
SELECT region, count() FROM anonymized_users GROUP BY region;

-- Средний чек по возрастным группам
SELECT age_group, avg(tx_amount) FROM anonymized_users GROUP BY age_group ORDER BY age_group;

-- Поиск по email невозможен без знания соли
SELECT * FROM anonymized_users WHERE email_hash = hex(SHA256(concat('wrongsalt',
'user@example.com')));

```

2.3.5.2. Контрольные вопросы

- Почему `DROP TABLE users_etl` даёт более однозначный момент уничтожения ПДн, чем завершённая мутация с `is_done = 1`?
- Субъект ПДн запросил удаление своих данных уже после того, как `users_etl` была удалена. Где ещё могут храниться его данные и что нужно проверить?

2.3.5.3. Когда уместно, когда нет

Уместно	Не уместно
Обезличивание ПДн с требованием подтверждения уничтожения	Частые обновления операционных данных — накладно держать две таблицы
Миграция и трансформация больших объёмов	Когда нужен live UPSERT без ETL-задержки
Нужна независимая аналитическая витрина поверх сырых данных	Схема источника часто меняется — поддержка трансформации становится дорогой

Таблица 14. ETL-пайплайн: применимость

2.3.6. Проверка k-анонимности

Обезличивание убрало прямые идентификаторы — ФИО, email, телефон. Но в таблице остались косвенные: возрастная группа и регион. Если какая-то комбинация этих двух полей встречается в таблице всего один раз — такую запись можно однозначно сопоставить с конкретным человеком, даже без имени.

k-анонимность — это измеримая гарантия: каждая комбинация квази-идентификаторов должна встречаться в датасете не менее k раз. При k=1 запись уникальна и уязвима; при k=5 злоумышленник не может сузить круг кандидатов меньше пяти человек.

Цель этого раздела — посчитать достигнутое k для `anonymized_users` и при необходимости поднять его до 5.

2.3.6.1. Задание

1. Посмотреть на распределение групп:

```

SELECT
    age_group,

```

```

    region,
    count() AS group_size
FROM anonymized_users
GROUP BY age_group, region
ORDER BY group_size ASC
LIMIT 20;

```

2. Найти минимальный размер группы — это и есть достигнутое k:

```

SELECT min(group_size) AS k_achieved
FROM (
    SELECT age_group, region, count() AS group_size
    FROM anonymized_users
    GROUP BY age_group, region
);

```

3. Если $k_achieved < 5$ — устранить уязвимые группы одним из способов:

- **Укрупнение бакетов:** пересоздать `anonymized_users` с более широкими диапазонами (например, «20–29» → «20–39») через `INSERT INTO ... SELECT` с изменённой формулой возраста.
- **Подавление:** удалить строки малых групп через `ALTER TABLE anonymized_users DELETE WHERE ....` Это мутация, но здесь она безопасна — `anonymized_users` не содержит ПДн, только производные атрибуты.

4. Повторить проверку до достижения $k \geq 5$.

2.3.6.2. Контрольные вопросы

- По фактическому результату `SELECT age_group, region, count()` посчитайте число уникальных комбинаций квази-идентификаторов. Какой верхний теоретический предел для k в этом датасете? Совпал ли он с реальным результатом?
- Почему удаление строк через `ALTER DELETE` безопасно именно для `anonymized_users`, хотя мы выяснили, что мутации не дают быстрой гарантии уничтожения?

2.3.7. Итоговое сравнение

Критерий	Мутация	RMT	ETL
Скорость	Медленно на больших объёмах	Быстро	Зависит от объёма
Консистентность во время операции	Нет	Нет (до merge)	Да
Момент уничтожения ПДн	<code>is_done = 1 + old_parts_lifetime (8 мин)</code>	<code>OPTIMIZE FINAL + old_parts_lifetime;</code> сигнал — <code>system.merges</code>	<code>DROP TABLE</code> (атомарно)
Откат	Невозможен	Невозможен	Возможен до DROP
Типичный сценарий	Разовое исправление	Обновление операционных данных	Обезличивание, миграция

Таблица 15. Сравнение вариантов обезличивания

2.4. Лабораторная работа 3: Аудит логов безопасности

2.4.1. Цель

Построить систему аудита security-событий на базе ClickHouse. Центральная задача — создать Materialized View для автоматической агрегации и на конкретном сценарии убедиться, что агрегат обновляется при каждом INSERT без ручного обновления. Побочная задача — увидеть, как append-only природа ClickHouse работает **в пользу** безопасности: удалить следы из логов сложнее, чем в OLTP-системе.

2.4.2. Часть 1: Схема данных и генерация событий

1. Создать таблицу security_events:

```
CREATE TABLE security_events (  
    timestamp    DateTime,  
    event_type   String,    -- 'login', 'access', 'privilege_change'  
    source_ip    String,  
    user_id      UInt32,  
    status       String,    -- 'success', 'failed', 'denied'  
    details      String  
) ENGINE = MergeTree()  
PARTITION BY toYYYYMM(timestamp)  
ORDER BY (event_type, source_ip, timestamp);
```

Ключ сортировки начинается с event_type (низкая кардинальность), затем source_ip, затем timestamp — это соответствует типичным фильтрам аналитических запросов.

2. Наполнить таблицу 100К событиями за «3 месяца». Среди событий должны быть:

- логины (успешные и неуспешные), включая серии brute-force с одного IP;
- доступ к ресурсам (разрешённый и запрещённый);
- изменения привилегий.

Генератор можно написать на Python или использовать INSERT ... SELECT с rand().

2.4.3. Часть 2: Materialized View — автоматическая агрегация

Создаем Materialized View для подсчёта неудачных логинов по IP и минуте, а затем убедимся, что агрегат обновляется автоматически при каждой вставке.

2.4.3.1. Задание

1. Создать целевую таблицу для агрегата:

```
CREATE TABLE failed_logins_per_minute (  
    minute    DateTime,  
    source_ip String,  
    attempts  UInt64  
) ENGINE = SummingMergeTree()  
ORDER BY (source_ip, minute);
```

Движок SummingMergeTree выбран потому, что при merge он суммирует attempts по ключу (source_ip, minute). Это корректно объединяет частичные агрегаты из разных блоков вставки.

2. Создать Materialized View:

```
CREATE MATERIALIZED VIEW mv_failed_logins  
TO failed_logins_per_minute  
AS SELECT  
    toStartOfMinute(timestamp) AS minute,  
    source_ip,  
    count()                    AS attempts  
FROM security_events  
WHERE status = 'failed' AND event_type = 'login'  
GROUP BY minute, source_ip;
```

3. Обязательный сценарий «было → вставили → стало»:

Выбрать IP-адрес, от которого пока не было неудачных логинов (или использовать новый, например '203.0.113.99').

Шаг 1. Убедиться, что в агрегате для этого IP пусто:

```
SELECT * FROM failed_logins_per_minute
WHERE source_ip = '203.0.113.99';
-- Пустой результат
```

Шаг 2. Вставить серию неудачных логинов:

```
INSERT INTO security_events VALUES
('2026-01-15 10:05:01', 'login', '203.0.113.99', 0, 'failed', 'brute-force attempt 1'),
('2026-01-15 10:05:02', 'login', '203.0.113.99', 0, 'failed', 'brute-force attempt 2'),
('2026-01-15 10:05:03', 'login', '203.0.113.99', 0, 'failed', 'brute-force attempt 3'),
('2026-01-15 10:05:04', 'login', '203.0.113.99', 0, 'failed', 'brute-force attempt 4'),
('2026-01-15 10:05:05', 'login', '203.0.113.99', 0, 'failed', 'brute-force attempt 5');
```

Шаг 3. Проверить агрегат — без ручного REFRESH или пересчёта:

```
SELECT * FROM failed_logins_per_minute
WHERE source_ip = '203.0.113.99';
-- minute: 2026-01-15 10:05:00, source_ip: 203.0.113.99, attempts: 5
```

Агрегат пополнился автоматически. Именно в этом состоит механизм MV в ClickHouse: при INSERT в исходную таблицу результат трансформации записывается в целевую таблицу без участия пользователя.

2.4.3.2. Контрольные вопросы

- Что произойдёт, если выполнить ещё один INSERT с тремя неудачными логинами от того же IP в ту же минуту? Сколько строк появится в failed_logins_per_minute до и после фонового merge?
- Почему для целевой таблицы выбран SummingMergeTree, а не обычный MergeTree? Что изменилось бы в результатах запросов?
- Если удалить строки из security_events через ALTER TABLE ... DELETE, изменятся ли данные в failed_logins_per_minute? Почему?

2.4.4. Часть 3: Brute-force detection — запрос к сырым данным vs. агрегат

2.4.4.1. Задание

1. Написать запрос к **сырой таблице**, который находит IP-адреса с более чем 10 неудачными логинами за любое 5-минутное окно:

```
SELECT
    source_ip,
    toStartOfFiveMinutes(timestamp) AS window,
    count() AS attempts
FROM security_events
WHERE status = 'failed' AND event_type = 'login'
GROUP BY source_ip, window
HAVING attempts > 10
ORDER BY attempts DESC;
```

2. Написать **аналогичный** запрос к агрегату failed_logins_per_minute. Поскольку агрегат хранит данные по минутам, для 5-минутного окна потребуется дополнительная группировка:

```
SELECT
    source_ip,
    toStartOfFiveMinutes(minute) AS window,
    sum(attempts) AS total_attempts
FROM failed_logins_per_minute
GROUP BY source_ip, window
HAVING total_attempts > 10
ORDER BY total_attempts DESC;
```

3. Сравнить результаты обоих запросов — они должны совпадать.
4. Обсудить: при каком объёме данных разница в производительности между запросами станет ощутимой? Почему на 100К строках ClickHouse может отвечать одинаково быстро в обоих случаях?

2.4.4.2. Контрольные вопросы

- В каком из двух запросов можно добавить фильтр по `user_id`? Почему агрегат этого не позволяет?
- Какой запрос вы бы использовали в дашборде, который обновляется каждые 30 секунд? Какой — для расследования конкретного инцидента?

2.4.5. Часть 4: Границы применимости Materialized View

В части 2 мы увидели, как MV автоматически пополняет агрегат при каждом INSERT. Естественное желание — пойти дальше и встроить в MV пороговую логику: пусть MV сама фиксирует алерт, когда число неудачных логинов превышает порог. ClickHouse не выдаст ошибку — запрос синтаксически корректен, MV создастся, вставки пройдут. Но результат будет неправильным.

2.4.5.1. Задание

Задача: «Автоматически фиксировать brute-force — записывать в таблицу алертов IP-адреса с более чем 20 неудачными логинами за минуту.»

1. Создать целевую таблицу и MV с HAVING:

```
CREATE TABLE brute_force_alerts (  
    minute    DateTime,  
    source_ip String,  
    attempts  UInt64  
) ENGINE = MergeTree()  
ORDER BY (source_ip, minute);  
  
CREATE MATERIALIZED VIEW mv_brute_force_alerts  
TO brute_force_alerts  
AS SELECT  
    toStartOfMinute(timestamp) AS minute,  
    source_ip,  
    count() AS attempts  
FROM security_events  
WHERE status = 'failed' AND event_type = 'login'  
GROUP BY minute, source_ip  
HAVING attempts > 20;
```

Синтаксических ошибок нет. MV создана.

2. Вставить 30 неудачных логинов от одного IP в одну минуту, но тремя батчами по 10:

```
INSERT INTO security_events  
SELECT toDate('2026-02-01 15:00:00') + number,  
       'login', '198.51.100.1', 0, 'failed', ''  
FROM numbers(10);  
  
INSERT INTO security_events  
SELECT toDate('2026-02-01 15:00:10') + number,  
       'login', '198.51.100.1', 0, 'failed', ''  
FROM numbers(10);  
  
INSERT INTO security_events  
SELECT toDate('2026-02-01 15:00:20') + number,  
       'login', '198.51.100.1', 0, 'failed', ''  
FROM numbers(10);
```

3. Проверить таблицу алертов:


```
SELECT * FROM brute_force_alerts
WHERE source_ip = '198.51.100.1';
```

Результат: **пустая таблица**. Ни одного алерта, хотя 30 неудачных логинов за минуту — это очевидный brute-force.

4. Убедиться, что события действительно есть — в сырой таблице все 30 строк на месте:

```
SELECT count() FROM security_events
WHERE source_ip = '198.51.100.1'
AND status = 'failed' AND event_type = 'login';
-- 30
```

2.4.5.2. Почему так произошло

ClickHouse не допустил ошибку — он сделал ровно то, что написано в определении MV. Проблема в том, что MV обрабатывает каждый INSERT изолированно. Каждый батч содержал 10 событий, HAVING attempts > 20 не сработал ни в одном из трёх — и алерт не был создан.

С точки зрения ClickHouse всё работает корректно. Но с точки зрения задачи обнаружения brute-force результат ложноотрицательный — атака произошла, а алерта нет.

Это не экзотический крайний случай. В реальных системах события приходят непрерывным потоком и попадают в ClickHouse множеством мелких батчей. MV с HAVING внутри будет систематически пропускать атаки, размазанные по нескольким вставкам.

2.4.5.3. Правильный подход

MV должна считать агрегаты **без порога**. Именно это делает mv_failed_logins из части 2 — она записывает в failed_logins_per_minute каждый блок без фильтрации, а SummingMergeTree объединяет частичные суммы при merge.

Проверку порога выполняет внешний запрос к уже готовому агрегату:

```
SELECT source_ip, minute, sum(attempts) AS total
FROM failed_logins_per_minute
WHERE minute >= now() - INTERVAL 5 MINUTE
GROUP BY source_ip, minute
HAVING total > 20
ORDER BY total DESC;
```

Такой запрос видит **все** вставки за минуту, а не отдельный батч, и корректно обнаруживает brute-force.

2.4.5.4. Контрольные вопросы

- Если бы все 30 событий были вставлены одним INSERT, сработал бы алерт? Должна ли корректность обнаружения атаки зависеть от того, сколько INSERT-ов использовал отправитель?
- Приведите пример другой задачи, где HAVING или LIMIT в MV даст ложный результат по той же причине.

2.4.6. Часть 5: Append-only и границы его полезности для безопасности

В ClickHouse нет привычного DELETE — удаление реализовано как мутация, которая перезаписывает данные в фоне и оставляет записи в системных таблицах. Это свойство архитектуры хранения, а не полноценный механизм защиты. Тем не менее оно создаёт два конкретных эффекта, полезных для аудита.

2.4.6.1. Задание

1. Попытаться удалить вставленные brute-force события из security_events:

```
ALTER TABLE security_events
DELETE WHERE source_ip = '203.0.113.99';
```

2. Проверить, что мутация зафиксирована:

```
SELECT command, is_done, create_time
FROM system.mutations
WHERE table = 'security_events'
ORDER BY create_time DESC LIMIT 5;
```


Даже после завершения мутации (`is_done = 1`) запись о ней остаётся — видно, **что** было удалено и **когда**.

3. Найти запись об удалении в `system.query_log`:

```
SELECT user, query, event_time
FROM system.query_log
WHERE query LIKE '%DELETE%security_events%'
ORDER BY event_time DESC LIMIT 5;
```

Видно, **кто** выполнил удаление.

4. Проверить, что агрегат в `failed_logins_per_minute` **не изменился**:

```
SELECT * FROM failed_logins_per_minute
WHERE source_ip = '203.0.113.99';
```

Строки из исходной таблицы удалены, но в целевой таблице MV агрегат на месте. Это второй эффект: целевая таблица MV — независимый объект, удаление из source её не затрагивает.

2.4.6.2. Где это помогает, а где нет

Append-only архитектура повышает **стоимость** заметания следов, но не делает его невозможным:

Что ClickHouse даёт	Чего он не даёт
DELETE оставляет запись в <code>system.mutations</code>	<code>system.mutations</code> — обычная таблица; пользователь с правами <code>admin</code> может её очистить
<code>system.query_log</code> фиксирует все запросы	<code>system.query_log</code> — MergeTree с TTL; записи удаляются по истечении срока
MV-агрегат не удаляется вместе с source	Целевую таблицу MV тоже можно DROP
Мутация — тяжёлая операция, заметная по нагрузке	TRUNCATE TABLE и DROP TABLE выполняются мгновенно и без мутаций

Таблица 16. Append-only: возможности и ограничения

В реальных системах журналы безопасности отправляют на **внешнее** хранилище с гарантией неизменяемости (WORM, S3 с Object Lock) именно потому, что ни одна СУБД сама по себе не является tamper-proof. ClickHouse — хороший инструмент для **анализа** логов, но не для их **защиты от уничтожения**.

2.4.6.3. Контрольные вопросы

- Пользователь с ролью `admin` из лабораторной 1 выполняет `TRUNCATE TABLE security_events`, а затем `TRUNCATE TABLE system.query_log`. Останутся ли следы этих действий? Где?
- Какие организационные или инфраструктурные меры (за пределами ClickHouse) нужны, чтобы гарантировать неизменяемость журнала безопасности?

2.4.7. Часть 6: Обнаружение аномальной активности

Задача: найти пользователей с нетипично большим количеством действий — более 3σ от среднего по всем пользователям.

2.4.7.1. Задание

1. Написать запрос, который считает число действий каждого пользователя и сравнивает с общим средним:

```
SELECT user_id, actions FROM (
    SELECT
        user_id,
        count() AS actions,
        avg(actions) OVER () AS mean_actions,
        stddevPop(actions) OVER () AS std_actions
    FROM (
        SELECT user_id, count() AS actions
```

```

        FROM security_events
        GROUP BY user_id
    )
)
WHERE actions > mean_actions + 3 * std_actions
ORDER BY actions DESC;

```

2. Объяснить: почему эта задача не подходит для Materialized View? (Подсказка: среднее и стандартное отклонение зависят от **всех** пользователей и пересчитываются при каждой новой вставке.)

2.4.7.2. Контрольные вопросы

- Что произойдёт с порогом $\text{mean} + 3\sigma$, если в систему добавится один пользователь с аномально высокой активностью? Как это повлияет на обнаружение **других** аномалий?
- Можно ли использовать MV для подсчёта числа действий по пользователям (без порога), а пороговую проверку выполнять запросом? Чем это похоже на подход из части 4?

2.4.8. Часть 7 (повышенная сложность): Дополнительные задачи

Эта часть предназначена для углублённого изучения. Основные задачи лабораторной — части 1–6.

1. **Эскалация привилегий:** найти цепочки «логин → изменение роли → доступ к ресурсу» от одного пользователя за короткий промежуток (менее 10 минут).
2. **Дополнительный MV:** создать второй Materialized View для другого агрегата (например, количество событий по типу за час) и повторить сценарий «было → вставили → стало».

2.5. Лабораторная работа 4: Private Set Intersection

2.5.1. Цель

Реализовать упрощённый протокол Private Set Intersection (PSI) на основе Diffie-Hellman: две «организации» находят общих клиентов, не раскрывая свои полные базы.

2.5.2. Часть 1: Подготовка данных

В этой лабораторной важны две вещи: контролируемое пересечение и разделение владения данными. Поэтому наборы данных должны быть похожими по размеру, но не совпадать полностью. Для воспроизводимости удобно сделать пересечение детерминированным, например 30K общих email из 100K.

2.5.2.1. Задание

1. Создать две таблицы с одинаковой структурой:

```
CREATE TABLE org_a_clients (  
    client_id UInt32,  
    email      String,  
    segment    LowCardinality(String)  
) ENGINE = MergeTree()  
ORDER BY email;
```

```
CREATE TABLE org_b_clients (  
    client_id UInt32,  
    email      String,  
    segment    LowCardinality(String)  
) ENGINE = MergeTree()  
ORDER BY email;
```

ORDER BY email не нужен для PSI как такового, но удобен для контрольного JOIN и последующего поиска расхождений.

2. Заполнить org_a_clients 100K строк:

```
INSERT INTO org_a_clients  
SELECT  
    number + 1 AS client_id,  
    concat('client', toString(number), '@example.com') AS email,  
    ['retail', 'b2b', 'vip'][number % 3 + 1] AS segment  
FROM numbers(100000);
```

3. Заполнить org_b_clients так, чтобы пересечение было частичным и заранее известным:

```
INSERT INTO org_b_clients  
SELECT  
    number + 1 AS client_id,  
    if(  
        number < 30000,  
        concat('client', toString(number), '@example.com'),  
        concat('client', toString(number + 200000), '@example.com')  
    ) AS email,  
    ['retail', 'partner', 'vip'][number % 3 + 1] AS segment  
FROM numbers(100000);
```

В таком варианте первые 30K email совпадают с организацией А, а остальные 70K гарантированно различны.

4. Подготовить таблицу для результатов PSI:

```
CREATE TABLE psi_results (  
    run_id      String,  
    email       String,  
    discovered_by LowCardinality(String),  
    created_at   DateTime DEFAULT now()
```

```
) ENGINE = MergeTree()  
ORDER BY (run_id, email);
```

Таблица намеренно append-only: каждый запуск PSI фиксируется отдельно и не затирает предыдущие результаты.

5. Выполнить быструю проверку размеров:

```
SELECT count() FROM org_a_clients; -- 100000  
SELECT count() FROM org_b_clients; -- 100000  
  
SELECT count()  
FROM org_a_clients AS a  
INNER JOIN org_b_clients AS b USING email;  
-- 30000
```

Этот INNER JOIN нужен только как эталон для преподавателя и для финальной верификации. Сам протокол PSI не должен опираться на него.

2.5.2.2. Контрольные вопросы

- Почему для лабораторной важно заранее знать размер пересечения, а не генерировать случайные наборы?
- Если обе таблицы отсортированы по email, ускоряет ли это сам PSI-протокол, выполняемый в Python? Что именно это ускоряет, а что — нет?

2.5.3. Часть 2: Реализация PSI

Для учебной работы достаточно упрощённого DH-варианта в мультипликативной группе по модулю большого простого числа p . Это не production-реализация PSI, а модель, на которой удобно понять идею двойного «ослепления».

Схема протокола такая:

1. Организация А вычисляет $H(email)^a \bmod p$ для каждого своего email и отправляет этот список организации В.
2. Организация В вычисляет $H(email)^b \bmod p$ для своих email, дополнительно возводит полученные от А значения в степень b и возвращает результат.
3. Организация А возводит значения В в степень a и сравнивает полученные маркеры. Совпадения соответствуют пересечению множеств.

Здесь $H(email)$ — детерминированное отображение email в число. В учебной версии достаточно $SHA-256(email) \bmod p$; полноценный hash-to-curve для этой лабораторной не обязателен.

2.5.3.1. Задание

Реализацию нужно разбить на два независимых скрипта — `org_a.py` и `org_b.py`. Каждый читает только свою таблицу из ClickHouse и знает только свой секрет. Данные двух организаций никогда не оказываются в одном процессе — это и есть ключевое свойство, которое лабораторная призвана показать на практике. Обмен между скриптами происходит через файлы с маркерами (списки чисел), как если бы это были сообщения по сети.

1. Реализовать в каждом скрипте:
 - загрузка email из своей таблицы (`org_a_clients` или `org_b_clients`);
 - хеширование email в число h in $[1, p - 1]$;
 - ослепление $\text{pow}(h, \text{secret}, p)$;
 - (только в `org_a.py`) сравнение маркеров и восстановление списка общих email.
2. Выбрать фиксированные параметры протокола — одинаковые в обоих скриптах:

```
P = 2**127 - 1 # или другое достаточно большое простое число
```

Секрет каждой организации (`A_SECRET`, `B_SECRET`) задаётся только в своём скрипте и в другой не передаётся.

3. Реализовать обмен по шагам: Формат обмена — TSV (допустим и CSV). В файлах передаются только маркеры.

- **Шаг 1.** `org_a.py` читает `org_a_clients` и вычисляет $H(email)^a \bmod p$. В `masked_a.tsv` сохраняется только список маркеров в фиксированном порядке. А локально хранит соответствие позиция $\rightarrow$ email; В его не получает.
- **Шаг 2.** `org_b.py` читает `masked_a.tsv` и возводит каждое значение в степень b . Так получается $(H(email)^a)^b \bmod p$. Возвращать эти значения нужно в том же порядке, в каком они пришли от А. Для своих email В отдельно вычисляет $H(email)^b \bmod p$ и сохраняет оба списка в `masked_b.tsv`.
- **Шаг 3.** `org_a.py` читает `masked_b.tsv` и доослепляет значения В своим секретом a . Так получается $(H(email)^b)^a \bmod p$. Совпадения между этим списком и $(H(email)^a)^b \bmod p$ из шага 2 образуют пересечение. Email восстанавливается по сохранённой позиции: если совпал i -й маркер из ответа В на список А, значит совпал i -й email из исходного списка А.

4. `org_a.py` записывает найденные совпадения в `psi_results`:

```
INSERT INTO psi_results (run_id, email, discovered_by) VALUES
('psi_run_001', 'client0@example.com', 'org_a');
```

`run_id` должен быть уникальным для каждого прогона, чтобы можно было сравнивать результаты нескольких запусков и замеров времени. Доступ на запись в `psi_results` есть только у организации А — организация В результат не видит.

5. В отчёте отдельно указать ограничение модели: в этой постановке результат пересечения фактически узнаёт организация А. Симметричный PSI, где итог доступен обеим сторонам без дополнительного раскрытия, требует более сложной схемы и в лабораторную не входит.

2.5.3.2. Контрольные вопросы

- Почему организация А не должна отправлять организации В сами email вместе со значениями $H(email)^a \bmod p$?
- Что сломается, если обе организации по ошибке используют один и тот же секрет вместо независимых a и b ?
- Почему для учебной модели допустимо использовать $\text{SHA-256}(email) \bmod p$, хотя это не эквивалентно полноценному hash-to-curve?

2.5.4. Часть 3: Верификация

Нужно проверить две вещи: корректность результата и цену приватности. PSI должен возвращать то же пересечение, что и обычный INNER JOIN, но с другим профилем затрат по процессору, сети и времени выполнения.

2.5.4.1. Задание

1. Проверить, что размер пересечения в `psi_results` совпадает с эталонным JOIN:

```
SELECT count()
FROM psi_results
WHERE run_id = 'psi_run_001';

SELECT count()
FROM org_a_clients AS a
INNER JOIN org_b_clients AS b USING email;
```

2. Проверить отсутствие расхождений по составу элементов:

```
SELECT count()
FROM
(
    SELECT email
    FROM psi_results
    WHERE run_id = 'psi_run_001'
```

```

) AS psi
FULL OUTER JOIN
(
    SELECT a.email
    FROM org_a_clients AS a
    INNER JOIN org_b_clients AS b USING email
) AS ref USING email
WHERE psi.email IS NULL OR ref.email IS NULL;

```

Корректная реализация должна вернуть 0: нет элементов, присутствующих только в одном из двух результатов.

3. Измерить накладные расходы протокола относительно прямого JOIN:
 - время обычного INNER JOIN в ClickHouse;
 - время чтения каждой организацией своих данных из ClickHouse (org_a.py и org_b.py замеряют отдельно);
 - время хеширования и возведения в степень в каждом скрипте;
 - время записи и чтения файлов с маркерами (симуляция сетевого обмена);
 - время записи результата в ClickHouse.
4. Сформулировать вывод по замерам. Ожидаемый результат: прямой JOIN окажется быстрее, потому что ClickHouse выполняет его внутри СУБД и без криптографических преобразований. PSI проигрывает по скорости, но выигрывает по модели раскрытия данных между сторонами.

2.5.4.2. Контрольные вопросы

- Почему сравнивать нужно не только количество строк, но и сами элементы пересечения?
- Какая часть времени в этой лабораторной обычно доминирует: SQL-JOIN, сетевой обмен с Python или криптографические операции? Почему?
- В каком случае PSI оправдан, если прямой JOIN почти наверняка быстрее?

2.5.5. Связь с архитектурой ClickHouse

В этой лабораторной граница между СУБД и прикладным уровнем становится ощутимой: ClickHouse хранит данные, Python выполняет криптографию.

1. MergeTree даёт каждой организации собственную физическую таблицу. Данные иммутабельны: другая сторона не может «подправить» чужую базу как в OLTP-сценарии с обычными UPDATE.
2. Row Policies из лабораторной 1 позволяют ограничить видимость таблиц org_a_clients и org_b_clients, чтобы каждая сторона читала только свой набор.
3. PSI выполняется **вне** ClickHouse: СУБД здесь отвечает за хранение и выборку, а не за саму криптографию. Это стандартный архитектурный подход: тяжёлая аналитическая БД плюс внешний вычислительный слой.
4. Результаты записываются в отдельную таблицу psi_results, а не изменяют исходные данные. Такой append-only подход совместим с аудитом: каждый запуск можно привязать к run_id, времени выполнения и пользователю, который инициировал процедуру.